KERALA GOVERNMENT POLYTECHNIC COLLEGE KOZHIKODE-5

DEPARTMENT OF COMPUTER ENGINEERING

2020-2023

PROJECT REPORT

ON

DriveMate

VEHICLE ASSISTANCE APP WITH AI-BASED ENGINE COMPLAINT PREDICTION BY ENGINE SOUND RECOGNITION

Submitted by

RAHEEMUDHEEN M A 20131112

Guided by

SHIHABUDHEEN K M

KERALA GOVERNMENT POLYTECHNIC COLLEGE KOZHIKODE-5

Department of Computer Engineering

CERTIFICATE

This is to certify that Project entitled **DRIVEMATE** *submitted by* **RAHEEMUDHEEN M A [Reg.No : 20131112]** *to the* **Department of Computer Science and Engineering, Kerala Government Polytechnic College Kozhikode-5** *, in partial fulfilment of the requirement for the award of Diploma under the Directorate of Technical Education, Government of kerala is a bonafide record of the work carried out by him.*

Mr. Shihabudheen K M	Mr. Shihabudheen K M
Head Of Department	**Guide**
Internal Examiner	**External Examiner**

ACKNOWLEDGEMENT

It is with great enthusiasm and learning spirit that I am bringing out the report of my Project. Here i would like to mark my token of gratitude to all those who influenced us during the period of my work. I would like to express my sincere thanks to **Mr. Suresh Kumar J S**, Principal, Kerala Government Polytechnic College, Kozhikode for the facilities provided here.

With immense pleasure i express sincere thanks to **Mr. Shihabudheen K M**, Head, Department of Computer Engineering, for the entire motivation and support. We also extend our heart-felt gratitude to my guide **Mr. Shihabudheen K M** and project co-ordinator **Dr. Shabeera TP**, Lecturer, Department of Computer Engineering, for the committed guidance, valuable suggestions. Also i extend my gratitude to all teachers in the Department of Computer Science and Engineering, Kerala Government Polytechnic College Kozhikode, for their support and inspiration.

ABSTRACT

Usage of internal combustion engines are drastically increased in our world. The petrol and diesel engine vehicles are Indispensable in our daily life. Majority of the population depends on this kind of vehicle for daily transportation. One of the major complaint of motor vehicles is engine complaints. Majority of the engine complaints can be identified from the engine sounds by a good experienced mechanic. The proposed application help us to find detection of abnormal sounds from a IC engine using unsupervised deep learning algorithm (KNN) thus predicting the failure of a engine early. In order to learn the sound of a vehicle engine with a deep learning algorithm, sound data recorded before and after the start-up of a vehicle engine and converted into a spectrogram images. The proposed application also helps to find mechanics easily and quickly. It is difficult to find mechanics nearby area wherever you are traveling. This system also helps to overcome this issue by providing mechanic details in one click. Here the locator allows you to search mechanics from different locations. Admin is allowed to access and manage mechanic details. This online mechanic locator reduces work and can easily find the mechanics from various location. Reduces your time and cost. Proposed system is accessed by three entities namely, Admin, Mechanic , Fuel pump and User. A mechanic can perform task such as viewing request received from users and can also send feedback to the admin.

TABLE OF CONTENTS

TITLE PAGE **i**

BONAFIDE CERTIFICATE **i**

ABSTRACT **iii**

LIST OF FIGURES **viii**

1 INTRODUCTION **1**

1.1 FEATURES OF THIS SYSTEM 2

2 SYSTEM ANALYSIS **3**

2.1 SYSTEM STUDY 3

2.1.1 EXISTING SYSTEM 3

2.1.2 PROPOSED SYSTEM 3

2.2 FEASIBILITY STUDY 4

2.2.1 OPERATIONAL FEASIBILITY 4

2.2.2 ECONOMIC FEASIBILITY 4

2.2.3 TECHNICAL FEASIBILITY 5

3 SYSTEM SPECIFICATION **6**

3.1 INTRODUCTION 6

3.1.1 PURPOSE 6

3.1.2 SCOPE 6

3.1.3 DEFINITIONS,ACRONYMS, AND ABBREVIATIONS 7

3.1.4 OVERVIEW 8

3.2 OVERALL DESCRIPTION 8

3.2.1 PRODUCT PERSPECTIVE 8

3.2.2 PRODUCT MODULES 9
3.2.3 PRODUCT FUNCTIONS 9
3.2.4 USER CHARACTERISTICS 12
3.2.5 PRINCIPLE ACTORS 12
3.2.6 GENERAL CONSTRAINTS 12
3.2.7 ASSUMPTION AND DEPENDENCIES 13
3.3 SPECIFIC REQUIREMENTS 14
3.3.1 FUNCTIONAL REQUIREMENTS 14
3.3.2 PERFORMANCE REQUIREMENTS 22
3.3.3 NON-FUNCTIONAL REQUIREMENTS 27
3.3.4 FUTURE EXTENSIONS 28

4 SYSTEM DESIGN 29
4.1 MODULE DESCRIPTION 29
4.1.1 ADMIN 29
4.1.2 USER 29
4.1.3 MECHANIC 30
4.1.4 FUEL PUMPS 30
4.2 DATA FLOW DIAGRAM 31
4.2.1 LEVEL 0 31
4.2.2 LEVEL 1.1(ADMIN) 31
4.2.3 LEVEL 1.2(USER) 32
4.2.4 LEVEL 1.3(MECHANIC) 32
4.2.5 LEVEL 1.4(FUEL) 32
4.2.6 LEVEL 1.4(FUEL) 33
4.3 DATA DATABASE DESIGN 34
4.3.1 Login 34
4.3.2 Location 34
4.3.3 Mechanic request 35
4.3.4 mechanic 35
4.3.5 Work request 35
4.3.6 Fuel Pump request 36

4.3.7 Fuel Pump 36
4.3.8 Fuel Delivery request 36
4.3.9 Vehicles 37
4.3.10 Mechanic Feed back 37
4.3.11 App Feed back 37
4.4 FORM DESIGN 38
4.4.1 Web & Android UI 38

5 SYSTEM IMPLEMENTATION 48
5.1 Methodology 48
5.1.1 Mel-Frequency Cepstral Coefficients (MFCC) Technique 48
5.1.2 K-Nearest Neighbors (KNN) Algorithm 49
5.2 Important Codes 50
5.2.1 Web codes 50
5.2.2 complaint prediction-KNN algorithm (Prediction.py) 53
5.2.3 web database connection (db.py) 58
5.2.4 Training Data (speech.py) 59
5.2.5 Android-data base connection (DataBasehelper.java) 61
5.2.6 MAinActivity(MAinActivity.java) 62

6 SYSTEM TESTING 65
6.1 UNIT TESTING 65
6.2 INTEGRATION TESTING 66
6.3 SYSTEM TESTING 67

7 CONCLUSION 68

8 FUTURE SCOPES 69

9 REFERENCES 70

LIST OF FIGURES

4.1	Level 0	31
4.2	Level 1.1	31
4.3	Level 1.2	32
4.4	Level 1.3	32
4.5	Level 1.4	33
4.6	Level 1.4	33
4.7	login	34
4.8	location	34
4.9	Mechanic request	35
4.10	mechanic	35
4.11	Work request	35
4.12	Fuel Pump request	36
4.13	Fuel Pump	36
4.14	Fuel Delivery request	36
4.15	Vehicles	37
4.16	Mechanic Feed back	37
4.17	App Feed back	37
4.18	WEB interfaces1	38
4.19	WEB interfaces2	39
4.20	WEB interfaces3	40
4.21	WEB interfaces4	41
4.22	Android interfaces 1	42
4.23	Android interfaces 2	43
4.24	Android interfaces 3	44
4.25	Android interfaces 4	45

4.26 Android interfaces 5 46
4.27 Android interfaces 6 47

CHAPTER 1

INTRODUCTION

DriveMate is a Innovative vehicle assistance application that leverages the power of artificial intelligence to predict engine problems through sound recognition technology. With DriveMate, drivers can quickly identify potential engine issues before they escalate, allowing them to take proactive measures to avoid costly repairs and prevent breakdowns on the road. The app is designed to detect and analyze engine sounds in real-time, using advanced algorithms to predict and diagnose potential problems. By providing drivers with this early warning system, DriveMate aims to improve safety, reduce repair costs, and enhance the overall driving experience for users. With DriveMate, drivers can have peace of mind knowing that their vehicle is always in top condition, thanks to the power of AI. This drive mate application also have features like nearby mechanic finding, On road fuel delivery, Vehicle document wallet.

1.1 FEATURES OF THIS SYSTEM

DriveMate is an innovative vehicle assistance application that is designed to provide drivers with an early warning system for potential engine problems. The app leverages the power of artificial intelligence and sound recognition technology to analyze engine sounds in real-time. This means that drivers can quickly identify potential issues before they escalate into more serious problems. By using advanced algorithms to predict and diagnose engine problems, DriveMate provides users with detailed reports and notifications about potential issues. These reports include information about the severity of the problem and recommended actions to take, such as taking the vehicle to a mechanic for repairs. This proactive approach to vehicle maintenance can help drivers avoid costly repairs and prevent breakdowns on the road.

DriveMate also offers additional features that can enhance the overall driving experience. For example, the app can track fuel efficiency and provide users with tips on how to improve their driving habits to save money on gas. This makes DriveMate a useful tool for both individuals and businesses with fleets of vehicles to manage. Overall, DriveMate provides drivers with peace of mind knowing that their vehicle is always in top condition. By improving safety, reducing repair costs, and enhancing the overall driving experience, DriveMate is a valuable tool for anyone who wants to take a proactive approach to vehicle maintenance and repair. And also in the case fuel delivary, it is not a cordinated system in our cities.

CHAPTER 2

SYSTEM ANALYSIS

2.1 SYSTEM STUDY

2.1.1 EXISTING SYSTEM

The traditional system for detecting potential engine problems was primarily done by experienced mechanics who could identify specific sounds or other signs of trouble with an engine. This process often involved a visual inspection of the engine and listening to its sounds. However, this approach has limitations and may not always be reliable, especially when it comes to predicting potential problems before they escalate into serious issues. With the advent of DriveMate, a new system is now available that leverages the power of artificial intelligence and sound recognition technology to predict engine problems in real-time. This system can be used by individual vehicle owners and businesses with fleets of vehicles to prevent costly repairs, improve safety, and enhance the overall driving experience. By providing drivers with early warning of potential issues before they become serious, DriveMate offers an innovative and effective alternative to the traditional system of engine problem detection.

2.1.2 PROPOSED SYSTEM

The proposed system in the context of DriveMate would be the AI-based engine complaint prediction app that uses sound recognition technology to detect and diagnose potential engine issues in real-time. The proposed system aims to improve the current system of engine problem detection by providing a reliable and accurate way of predicting potential problems before they become serious. By using advanced algorithms to analyze engine sounds, DriveMate can identify and diagnose issues that may not be detectable to the human ear, allow-

ing drivers to take proactive measures to prevent breakdowns and costly repairs.

The proposed system offers several key benefits over the traditional system, including improved accuracy, reliability, and speed of detection. By providing detailed reports and notifications about potential engine issues, DriveMate can help drivers stay on top of vehicle maintenance and prevent small issues from turning into major problems. Additionally, DriveMate can provide users with peace of mind by allowing them to monitor the health of their vehicle in real-time, reducing the risk of unexpected breakdowns on the road. And the integrated location based fuel delivery function can be a very use full feature for the vehicle owners who have a trouble of low fuel level at a mid journey

Overall, the proposed system of DriveMate offers a revolutionary way of predicting and diagnosing potential engine issues that can greatly improve the driving experience and reduce the costs associated with vehicle maintenance.

2.2 FEASIBILITY STUDY

Feasibility study is the test of system according to work ability impact on organization ability to meet user needs and effective use of resources

•Operational Feasibility

• Economic Feasibility

• Technical Feasibility

2.2.1 OPERATIONAL FEASIBILITY

Operation feasibility is used to check whether the project is operationally feasible or not. Our project is mainly different from the other system because of its web-support and sorting feature.So the measure for operational feasibility is something different from other system. Generally the operational feasibility is related to organization aspects.

2.2.2 ECONOMIC FEASIBILITY

Economical feasibility is the measure to determine the cost and benefit of the proposed system. A project is economical feasible which is under the estimated cost for its development. These benefits and costs may be tangible or intangible.Routes is the cost-effective project in which there is less possibility of intangible cost so there is no difficulty to determine the cost of the project.

2.2.3 TECHNICAL FEASIBILITY

Technical feasibility study is concerned with specifying equipment and software that will successfully satisfy the user requirement; the technical needs of the system may vary considerably.

All these technologies are easy to learn and can develop system very rapidly. After developing and deploying the system, any user can use the application.

CHAPTER 3

SYSTEM SPECIFICATION

3.1 INTRODUCTION

3.1.1 PURPOSE

The purpose of this system is to provide an innovative solution for predicting and diagnosing potential engine issues in vehicles. The existing system relies heavily on the experience of well-trained automobile mechanics to identify engine problems from sound, which can be unreliable and expensive. The proposed system, DriveMate, leverages the power of artificial intelligence and sound recognition technology to detect and diagnose potential engine issues in real-time, providing drivers with early warnings and detailed reports about potential problems.

The DriveMate app aims to improve the overall driving experience by reducing the risk of breakdowns and costly repairs, and by providing drivers with peace of mind that their vehicle is always in top condition. By analyzing engine sounds using advanced algorithms, DriveMate can identify and diagnose issues that may not be detectable to the human ear, allowing drivers to take proactive measures to prevent breakdowns and costly repairs.

Overall, the proposed system of DriveMate offers a revolutionary way of predicting and diagnosing potential engine issues that can greatly improve the driving experience and reduce the costs associated with vehicle maintenance.

3.1.2 SCOPE

The scope of the DriveMate project encompasses several aspects related to vehicle maintenance, fuel delivery, and convenience for vehicle owners. Here are the main scopes of the project:

1. Engine complaint prediction by engine sound recognition: The project aims to develop a system that can analyze the sound produced by a vehicle's engine and predict potential complaints or issues. By using advanced audio recognition technology, the system can detect abnormal engine sounds and provide early warnings to vehicle owners, allowing them to take proactive measures to prevent potential breakdowns or damages.

2. On-road fuel delivery : The project includes the implementation of a fuel delivery service that caters to the needs of vehicle owners. By leveraging technology and logistics, the system allows users to request fuel delivery directly to their location, eliminating the need to visit a fuel station. This service offers convenience and time-saving benefits for vehicle owners, especially in situations where refueling may be inconvenient or time-consuming.

3. Nearby mechanic finding: The DriveMate project aims to provide a feature that enables vehicle owners to find nearby mechanics or service centers easily. By utilizing location-based services and integrating with mapping platforms, the system can identify the nearest and most suitable mechanics based on the user's location, vehicle type, and specific requirements. This feature helps vehicle owners quickly locate assistance when facing unexpected breakdowns or requiring maintenance services.

4. Digital vehicle document wallet: The project includes the development of a digital wallet system that allows users to store and manage essential vehicle documents electronically. This feature eliminates the need for physical document storage and provides a secure and convenient way to access and organize important vehicle-related information, such as registration papers, insurance documents, service records, and more.

In summary, the main scopes of the DriveMate project revolve around engine complaint prediction, on-road fuel delivery, nearby mechanic finding, and digital vehicle document management. These scopes aim to enhance vehicle maintenance, convenience, and efficiency for vehicle owners, ultimately improving their overall driving experience.

3.1.3 DEFINITIONS,ACRONYMS, AND ABBREVIATIONS

DriveMate Vehicle Assistance App

1 ML: Mechine Learning

2 AI: Artificial Intelligence

3 KNN:K-Nearest Neighbors

4 MFCC:Mel Frequency Cepstral Coefficients

5 SRS:Software Requirements Specification

3.1.4 OVERVIEW

DriveMate is a project that focuses on vehicle maintenance, fuel delivery, and convenience for vehicle owners. While it does not directly relate to pest detection in agriculture, I can provide an overview of the project for you.

DriveMate aims to provide innovative solutions to enhance the overall experience of vehicle owners. It focuses on leveraging technology to predict engine complaints through engine sound recognition, enabling proactive maintenance measures to prevent breakdowns or damages. Additionally, the project includes an on-road fuel delivery service that allows users to request fuel directly to their location, saving time and offering convenience. DriveMate also facilitates the process of finding nearby mechanics or service centers for vehicle owners, ensuring quick assistance in case of unexpected breakdowns or maintenance needs. Finally, the project involves the development of a digital vehicle document keeping wallet, offering a secure and convenient way to store and manage essential vehicle documents electronically.

In summary, DriveMate aims to provide vehicle owners with advanced technological solutions for engine complaint prediction, on-road fuel delivery, nearby mechanic finding, and digital vehicle document management. These scopes enhance vehicle maintenance, convenience, and efficiency, ultimately improving the overall experience for vehicle owners.

3.2 OVERALL DESCRIPTION

3.2.1 PRODUCT PERSPECTIVE

The product perspective of DriveMate refers to how the DriveMate system fits within the larger context of the vehicle ownership and maintenance ecosystem and its users.

3.2.2 PRODUCT MODULES

This product consists of total 4 modules :

1. Admin (web).

2. Vevicle users (android).

3. Mechanic (android).

4. Fuel Pump (android).

3.2.3 PRODUCT FUNCTIONS

The application provides the following functions :

Class of use cases	Use cases	Description of use cases
Use cases related to installation	Installation	Creates & initializes working files
Use cases related to system authorization	Login	Login to DriveMate
Use cases related to Admin module	Add data set	Add data sets for engine sound recognition
	View Mechanics	View Mechanics and their details
	Accept or reject mechanic	Accept or reject new mechanic registration
	View Fuel pumps	View Fuel pumps and their details
	Accept or reject Fuel pump	Accept or reject new Fuel pump registration
	View App feedbacks	View feed backs about DriveMate application from users

Use case related to User module	Register	A new user can register in DriveMate app
	View & Edit profile	Signed user can view their profile and edit the details
	Engine complaint prediction	User can access the Ai-engine complaint prediction feature
	Find Mechanic	User can find nearest mechanic for any vehicle related works
	Mechanic feedback	User can give a feedback and give rating to mechanics
	Find fuel Pump	User can find nearby fuel pumps
	Order fuel	User can order fuel on road
	App feed back	User can give a feedback and give rating about the DriveMate application

Use case related to Mechanic module	Register	A new Mechanic can send registration request to Admin
	View & Edit profile	Signed Mechanic can view their profile and edit the details
	View request	View work requests from users
	Accept or reject request	Mechanic can either Accept or Reject Work requests from Users
	Update works	Mechanic can update the works after completion
	View feedbacks	Mechanic can view feedbacks from user

Use case related to Fuel pumps module	Register	A new Fuel pump can send registration request to Admin
	View & Edit profile	Signed Fuel pump can view their profile and edit the details
	View request	View Fuel Delivery requests from users
	Accept or reject request	Fuel pump can either Accept or Reject Work requests from Users
	Update works	Fuel pump can update the works after completion

3.2.4 USER CHARACTERISTICS

Users: The DriveMate system caters to four primary user groups: Admin, vehicle owners, fuel stations, and mechanics.

3.2.5 PRINCIPLE ACTORS

User Characteristics actors in AD are “User” , “Admin” and “Emergency unit”.

3.2.6 GENERAL CONSTRAINTS

There are several constraints that may affect the design and implementation of the DriveMate system.

1. Cost: Developing and implementing the DriveMate system may involve significant costs, including software development, hardware infrastructure, maintenance, and operational expenses. The cost factor could potentially limit accessibility to the system, particularly for vehicle owners with limited financial resources.

2. Technical Expertise: Effective utilization of the DriveMate system requires a certain level of technical expertise, such as understanding vehicle maintenance, sound recognition technology, and digital document management. Users, including vehicle owners, mechanics, and fuel station personnel, may need training or support to effectively use and navigate the system.

3. Access to Technology: The accessibility of the DriveMate system may be influenced by the availability of technology infrastructure, including mobile devices, internet connectivity, and compatible software platforms. In areas with limited access to technology or unreliable network connectivity, the adoption and usage of the DriveMate system may be challenging.

4. Environmental Variability: Environmental factors, such as varying road conditions, weather, and geographical characteristics, can impact the performance and accuracy of certain features of the DriveMate system. Additional sensors or data sources may be required to account for such variability and provide reliable results.

5. Ethical Concerns: The use of the DriveMate system, particularly in terms of data collection and storage, may raise ethical concerns related to privacy and data security. It is crucial to ensure that user data is protected and handled in compliance with relevant privacy regulations.

6. Integration and Compatibility: The DriveMate system may need to integrate with existing infrastructure, such as vehicle systems, fuel stations, and mapping platforms. Compatibility and integration challenges may arise, requiring careful planning and coordination with relevant stakeholders.

Considering these constraints during the design and implementation of the DriveMate system is crucial to ensure its accessibility, usability, and effectiveness for all user groups involved.

3.2.7 ASSUMPTION AND DEPENDENCIES

Assumptions and dependencies are key considerations when designing and implementing the DriveMate system.

1. Availability of Data: The DriveMate system relies on the availability of data, such as vehicle sound samples, fuel station information, mechanic locations, and digital vehicle documents. Assumptions are made that this data is available and accurate for the system to function effectively.

2. Infrastructure: The DriveMate system requires infrastructure, including mobile networks, internet connectivity, computer systems, and GPS services. Assumptions are made that the necessary infrastructure is in place and functioning properly to support the system's functionalities.

3. Standardized Information: DriveMate's functions, such as engine complaint prediction, nearby mechanic finding, and digital document management, may rely on standardized information, such as engine sound databases, mechanic service details, and document formats. Assumptions are made that these standards exist and are used consistently for seamless integration and accurate results.

4. User Engagement: The effectiveness of the DriveMate system relies on user engagement, feedback, and collaboration. Assumptions are made that users, including vehicle owners, mechanics, fuel stations, and administrators, are willing and able to actively engage with the system, provide feedback, and validate results.

5. Availability of Resources: The successful implementation and operation of the DriveMate system require resources, including funding, skilled personnel, and necessary equipment. Assumptions are made that these resources are available and sufficient to support the system's development, maintenance, and ongoing operations.

6. Legal and Regulatory Compliance: The DriveMate system needs to comply with applicable legal and regulatory requirements, such as data privacy, vehicle-related regulations, and intellectual property rights. Assumptions are made that the system is designed and implemented in compliance with these requirements to ensure legal and ethical use of user data and intellectual property.

Considering these assumptions and dependencies is essential for the effective design, devel-

opment, and implementation of the DriveMate system, ensuring its seamless integration, user acceptance, and compliance with relevant regulations and standards.

3.3 SPECIFIC REQUIREMENTS

3.3.1 FUNCTIONAL REQUIREMENTS

Use case related to Installation

Use Case 1: Installation

Actor: Vehicle User, Mechanic, fuelStation

Pre Condition: Internet connection available.

Main Scenario :

1. Actors initiates App installation program.
2. The program contain :
    - pycharam
    - Dreamweaver
    - Pyroid 6.4
    - Wampserver
    - SQLyog
    - Android Studio

3.System downloads the Pycharm,Dreamweaver, Android Studio, Pyroid 6.4,Wampserver and SQLyog .

4.. Activate the environment : Pycharm/open file location/scripts/cmd.

Alternate Scenario :

(a) Network failure.

(b) Installation aborted

Use cases related to system login:

Use Case 2: Login

Primary Actor: User, Mechanic, FuelPumps,Admin(web)

Pre Condition: Nil

Main Scenario :

1. Start the application. User prompted for login and password.

2. User gives the login and password.

3. System does authentication.

4. Main screen is displayed.

Alternate Scenario :

1. 4(a). Authorization fails :

a. 4(a)1. Prompt the user that he typed the wrong password

Use case related to Admin(web)

Use Case 3: Manage dataset

Primary Actor: Admin

Pre Condition: Nil

Main Scenario :

1. When user select Manage dataset option
2. System open file directory to select the data set folder to upload.
3. System save datasets into system

Alternate scenario:

4. Invalid data set

Use Case 4: View Mechanics

Primary Actor:Admin

Pre Condition: must have registered mechanic

Main Scenario:

1. Admin selects the option of view Mechanic

2. System displays name,address and phone number of the registered Mechanic and the feed backs of mechanic from users

Alternate scenario:

1.Database crash

Use Case 5 :Accept or reject mechanic

Primary Actor:Admin

Pre Condition: new Mechanic must sent a registration request

Main Scenario:

1.Admin select view request option

2. Mechanic can accept or reject regisration request from mechanic

Alternate scenario:

1.invalid data types

Use Case 6: View Fuel pumps

Primary Actor:Admin

Pre Condition: must have registered FuelPumps

Main Scenario :

1. Admin selects the option of view Fuel pumps
2. System displays name,address and phone number of the registered Fuel pumps and the feed backs of Fuel pumps from users

Alternate scenario:

Use Case 7: Accept or reject Fuel pump

Primary Actor:Admin

Pre Condition:new Fuel pump must sent a registration request

Main Scenario :

1.Admin select view request option

2. Admin can accept or reject regisration request from Fuel pump

Alternate scenario:

Use Case 8 : View App feedbacks

Primary Actor:Admin

Pre Condition: user must sent app feedbacks

Main Scenario :

1.User select App feed back option System shows the user's feedback about the DriveMate application

Use case related to User

Use Case 9:Register

Primary Actor:User

Pre Condition: nill

Main Scenario :

1. User can add name,place,phone, email, username,password for registration

Use Case 10:View or Edit profile

Primary Actor:User

Pre Condition:User logged in

Main Scenario:

1. User must select the profile option
2. System display the details of logined user
3. user can Edit the entered details

Alternate scenario:invalid data types

Use Case 11:Engine complaint prediction

Primary Actor:User

Pre Condition:User logged in

Main Scenario :

1. User select the Ai complaint predict option .

1. System will show the optin to upload a prerecorded Audio file,or recording button to live record the Audio .

Alternate scenario:

1. invalid Audio format(Audio format other than .WAV) .

Use Case 12:Find Mechanic

Primary Actor:User

Pre Condition:User logged in

Main Scenario:

1. User select find mechanic from user home page .

2.system will list out nearby mechanic in order of distance

3.user can selcect desirable mechanic

4.system will show field to enter a brief about the work and sent work request

5.user can directly call the mechanic

Use Case 13:Mechanic feedback

Primary Actor:User

Pre Condition:User logged in

Main Scenario :

1. User can give a feed back and rating to Mechanic.

Use Case 14:Find fuel Pump

Primary Actor:User

Pre Condition:User logged in

Main Scenario :

1. User select find fuel Pump from user home page .

2.system will list out nearby fuel Pump in order of distance

3.user can selcect desirable fuel Pump

4.system will show field to select quantity and fuel types

5.user will sent request to delivary the fuel

Use Case 14:App feed back

Primary Actor:User

Pre Condition:User logged in

Main Scenario :

1.User can give a feedback and give rating about the DriveMate application.

Use case related to Mechanic

Use Case 15:Register

Primary Actor:Mechanic

Pre Condition: nill

Main Scenario :

1. Mechanic can add name,place,phone, email, username,password for requesting registration to admin

Alternate scenario:

1.invalid data types

Use case 16:View & Edit profile

Primary Actor:Mechanic

Pre Condition:Mechanic logged in

Main Scenario :

1. Mechanic must select the profile option
2. System display the details of logined Mechanic
3. Mechanic can Edit the entered details

Alternate scenario:

1.invalid data types

Use case 17:View request

Primary Actor:Mechanic

Pre Condition:Mechanic logged in

Main Scenario :

1. Mechanic select the view work request option
2. System display the work requests from users

Alternate scenario:

1.no work requests from users

Use case 18:Accept or reject request

Primary Actor:Mechanic

Pre Condition:Mechanic logged in

Main Scenario :

1. Mechanic select a request from list
2. System display the detail of work from users

3.Mechanic select location to view user location in google maps,system navigate to google map app

4.Mechaic can either accept or reject work request

Use case 19:Update works

Primary Actor:Mechanic

Pre Condition:Mechanic logged in

Main Scenario :

1. Mechanic select the update work option
2. System display the accepted works from users

3.user update the work as completed after the job done

Alternate scenario:

1.no work requests from users

Use case 20:View feedbacks

Primary Actor:Mechanic

Pre Condition:Mechanic logged in

Main Scenario :

1. Mechanic select the feed back option
2. System display the ratings and feedbacks users

Alternate scenario:

1.no workfeedback from users

Use case related to Fuel pumps

Use Case 21: Register

Primary Actor:Fuel pumps

Pre Condition: nill

Main Scenario :

1. Fuel pumps can add name,place,phone, email, username,password for requesting registration to admin

Alternate scenario:

1.invalid data types

Use case 22: View & Edit profile

Primary Actor:Fuel pumps

Pre Condition:Fuel pumps logged in

Main Scenario :

1. Fuel pumps select the profile option
2. System display the details of logined Fuel pumps
3. Fuel pumps can Edit the entered details

Alternate scenario:

1.invalid data types

Use case 23: View request

Primary Actor:Fuel pumps

Pre Condition:Fuel pumps logged in

Main Scenario :

1. Fuel pumps select the view delivery request option
2. System display the requests from users

Alternate scenario:

1.no delivery requests from users

Use case 24: Accept or reject request

Primary Actor:Fuel pumps

Pre Condition:Fuel pumps logged in

Main Scenario :

1. Fuel pump select a request from list
2. System display the detail of delivery from users

3.Fuel pump select location to view user location in google maps,system navigate to google map app

4.Fuel pump can either accept or reject delivery request

Use case 25: Update works

Primary Actor:Fuel pump

Pre Condition:Fuel pump logged in

Main Scenario :

1. Fuel pump select the update work option
2. System display the accepted delivery from users

3.Fuel pump update the delivery as completed after the delivery done

Alternate scenario:

1.no work requests from users

3.3.2 PERFORMANCE REQUIREMENTS

HARDWARE AND SOFTWARE REQUIREMENTS

HARDWARE REQUIREMENTS

- Processor : Intel(R) Core(TM) i3-3110M CPU@2.40GHZ(or above)
- Memory(RAM) : 4.00 GB
- Hard disk : 500 GB
- Input devices : Keyboard,touch pad
- Output devices :Monitor,Andriod Phone with basic functionality
- Android smartphone : with minimum android version of 8

SOFTWARE REQUIREMENTS

Server Side (Web Application)

• Frontend : Phychram (using Python)

• Backend : MySQL

• Web Server : Flask

• OS : Windows, Mac OS, Linux etc.

Mobile Side

• Frontend : Android studio

• Language : java

• OS : Android/iOs

• IDE : Visual Studio Code

FRONT END

Andriod Studio

Android Studio is an integrated development environment (IDE) specifically designed for Android app development. It provides a comprehensive set of tools and features that help developers build, test, and debug Android applications efficiently. Here are some advantages of using Android Studio:

1. Robust Development Environment: Android Studio offers a powerful and feature-rich development environment tailored for Android app development. It provides a user-friendly interface with various tools and editors that streamline the development process.

2. Extensive SDK and Libraries: Android Studio comes with the Android Software Development Kit (SDK), which includes a wide range of APIs, libraries, and resources to build Android apps. It provides access to platform-specific features and functionality, allowing developers to create rich and immersive applications.

3. Integrated Emulator and Device Testing: Android Studio includes an emulator that allows developers to test their apps on virtual devices with different screen sizes, resolutions, and Android versions. It also provides seamless integration with physical Android devices for real-time testing and debugging.

4. Gradle Build System: Android Studio uses the Gradle build system, which simplifies the management of dependencies, resources, and build configurations. Gradle provides flexibility and scalability, allowing developers to customize the build process to meet specific requirements.

5. Layout Editor and Design Tools: Android Studio offers a powerful layout editor that enables visual design of user interfaces (UI) for Android apps. It includes drag-and-drop functionality, XML editing, and previewing tools, making it easier to create and modify UI elements.

6. Code Assistance and Refactoring: Android Studio provides intelligent code completion, refactoring tools, and code templates that enhance productivity and code quality. It assists developers with suggestions, auto-completion, and error detection, helping to reduce errors and improve efficiency.

7. Profiling and Performance Analysis: Android Studio includes profiling and debugging tools that help identify and optimize app performance issues. Developers can analyze CPU usage, memory allocation, network activity, and more to ensure their apps run smoothly and efficiently.

8. Version Control Integration: Android Studio seamlessly integrates with version control systems like Git, allowing developers to manage source code, track changes, and collaborate with other team members.

9. Comprehensive Documentation and Support: Android Studio is backed by Google's extensive documentation and community support. Developers can access official guides, tutorials, and online resources to learn and troubleshoot Android development.

Overall, Android Studio provides a robust and efficient development environment for creating Android applications. It offers a range of tools, libraries, and features that support the development lifecycle, making it the preferred choice for Android app developers.

JAVA

Java was originally developed by James Gosling at Sun Microsystems and released in May 1995 as a core component of Sun Microsystems' Java platform. The original and reference implementation Java compilers, virtual machines, and class libraries were originally released by Sun under proprietary licenses. Principles There were five primary goals in the creation of the Java language: 1. It must be simple, object-oriented, and familiar.

2. It must be robust and secure.

3. It must be architecture-neutral and portable.

4. It must execute with high performance.

5. It must be interpreted, threaded, and dynamic.

IDE

Android Studio

Visual Studio Code, also commonly referred to as VS Code, is a sourcecode editor made by Microsoft for Windows, Linux and macOS. Features include support for debugging, syntax highlighting, intelligent code completion, snippets, code refactoring, and embedded Git. Users can change the theme, keyboard shortcuts, preferences, and install extensions that add additional functionality. In the Stack Overflow 2021 Developer Survey, Visual Studio Code was ranked the most popular developer environment tool, with 7010 they use it. Visual Studio Code was first announced on April 29, 2015, by Microsoft at the 2015 Build conference. A preview build was released shortly thereafter. On November 18, 2015, the source of Visual Studio Code was released under the MIT License, and made available on GitHub. Extension support was also announced. On April 14, 2016, Visual Studio Code graduated from the public preview stage and was released to the Web. Microsoft has released most of Visual Studio Code's source code on GitHub under the permissive MIT License, while the releases by Microsoft are proprietary freeware.

Macromedia Dreamweaver IDE

Macromedia Dreamweaver, now known as Adobe Dreamweaver, is an integrated development environment (IDE) specifically designed for web development. It provides a comprehensive set of tools and features that assist developers in creating and designing websites and web applications. Dreamweaver offers a visual design interface combined with a powerful code editor, making it suitable for both beginner and advanced developers.One of the key features of Dreamweaver is its WYSIWYG (What You See Is What You Get) design view, which allows developers to visually design web pages by dragging and dropping elements onto the canvas. This visual design mode provides a real-time preview of how the web page will appear in a web browser, making it easier to create visually appealing layouts.Dreamweaver also includes a code editor with syntax highlighting and code suggestion features for various web languages such as HTML, CSS, JavaScript, and PHP. It offers advanced code editing capabilities, including code folding, code snippets, and code validation, helping developers write clean and error-free code.The IDE provides built-in support for popular web technologies and frameworks, including Bootstrap, jQuery, and WordPress. It simplifies the integration of these technologies into web projects by providing code templates, component libraries, and easy-to-

use tools for managing and updating dependencies.Another notable feature of Dreamweaver is its integration with version control systems such as Git. Developers can manage code repositories, perform version control operations, and collaborate with team members directly within the IDE. Dreamweaver also offers powerful features for testing and debugging websites. It provides built-in browser compatibility testing, allowing developers to check how their websites look and function across different browsers and devices. The Live View feature enables developers to see how dynamic content, such as JavaScript interactions, behaves in real-time.Additionally, Dreamweaver supports server-side development with features like database integration, server behaviors, and dynamic content creation. It simplifies the process of connecting to databases, querying data, and generating dynamic web pages.Overall, Macromedia Dreamweaver (now Adobe Dreamweaver) has been a popular IDE for web development due to its combination of visual design capabilities and robust code editing features. It provides a comprehensive set of tools and integrations that streamline the web development workflow and cater to the needs of both beginners and experienced developers.

BACK END

MySQL

MySQL is a relational database management system (RDBMS) developed by Oracle that is based on structured query language (SQL). A database is a structured collection of data. It may be anything from a simple shopping list to a picture gallery or a place to hold the vast amounts of information in a corporate network. In particular, a relational database is a digital store collecting data and organizing it according to the relational model. In this model, tables consist of rows and columns, and relationships between data elements all follow a strict logical structure. An RDBMS is simply the set of software tools used to actually implement, manage, and query such a database. MySQL is integral to many of the most popular software stacks for building and maintaining everything from customer-facing web applications to powerful, data-driven B2B services. Its open-source nature, stability, and rich feature set, paired with ongoing development and support from Oracle, have meant that internet-critical organizations such as Facebook, Flickr, Twitter, Wikipedia, and YouTube all employ MySQL backends. In a simple manner, SQL is a non-procedural, English-like language that processes data in groups of records rather than one record at a time.

WHY SQL USED IN THIS SYSTEM

SQL server can be used to develop both simple and complex database. Its wonderfully designed built-in wizard makes allow look like a product that any one can use. Although it is true that the simplest of the access application can be produced without any thought of design and without a single line of code written by the user, most applications require at least some designing and customer codes. It can store more data.

3.3.3 NON-FUNCTIONAL REQUIREMENTS

Non-Functional requirements are the constraints or the requirements im-posed on the system. They specify the quality attribute of the software.

SECURITY

The system need to store the data in a secure location and store in a secure format. It requires a secure communication channel for the data, and the system needs to control the system's sessions.

CONCURRENCY and CAPACITY

System should be able to let a vehicle user can find their vehicle's critical engine complaints in advanced time.The system also provide the feature for finding nearest mechanic and fuel pumps.

RELIABILITY

End user satisfaction and acceptance is one of the key pillars that support a project success. As we are using Flask framework, we can implement a simple yet powerful system and thereby it enhances the system usability.

MAINTAINABILITY

The system need to be regularly checked for preventive and corrective maintenance and updation in data sets as it would come up with addition feature requirements or removing undesired events. Maintenance might specify scalability to grow and improve the system features and functionalities

USABILITY

End user satisfaction and acceptance is one of the key pillars that support a project success. As we are using Flutter framework, we can implement a simple yet powerful system and thereby it enhances the system usability

3.3.4 FUTURE EXTENSIONS

In the future, DriveMate could expand its capabilities by integrating with smart home devices, allowing users to receive real-time updates and notifications about their vehicle's status even when they are at home. Additionally, advanced predictive maintenance models could be developed by analyzing patterns and trends across a fleet of vehicles, providing users with timely recommendations for service or part replacements. DriveMate could also introduce social features and community engagement, creating a platform where users can share their driving experiences and tips with each other. Furthermore, as the popularity of electric vehicles grows, DriveMate could expand its functionalities to include diagnostics and maintenance recommendations specific to EVs, catering to the unique needs of EV owners.

CHAPTER 4

SYSTEM DESIGN

4.1 MODULE DESCRIPTION

The complete project is divided into four modules. And the modularization is based on the type of the users in the system. The different modules based on the type of the users of the system are:

1. Admin module(web)

2. User module(android)

3. Mechanic module(android)

4. Fuel Pumps module(android)

4.1.1 ADMIN

1.Manage Datasets

2.Add ,view and delete Mechanics

3.Add,view and delete Fuel Pumps

4.View App feedbacks

4.1.2 USER

1.Create a new account

2.View and Edit profile

3.find nearby mechanic and request for their service

4.find nearby Fuelpumb and request for their service

5.Use utilize the Ai-engine complaint prediction feature

4.1.3 MECHANIC

1.Create a new account

2.View and Edit profile

3.View the work requests from user

4.Accept or reject the work requests

5.update accepted work

6.view feed backs from users

4.1.4 FUEL PUMPS

1.Create a new account

2.View and Edit profile

3.View the delivery requests from user

4.Accept or reject the delivery requests

5.update accepted delivery

4.2 DATA FLOW DIAGRAM

4.2.1 LEVEL 0

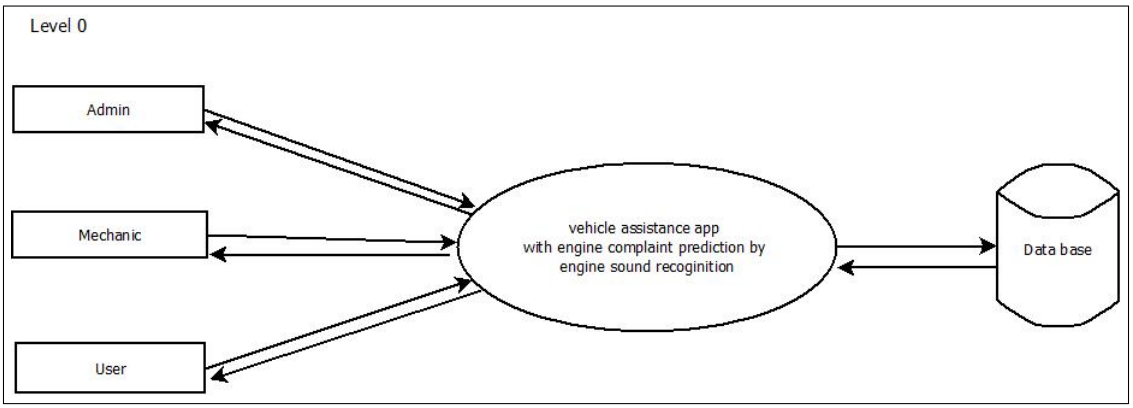


Figure 4.1: Level 0

4.2.2 LEVEL 1.1(ADMIN)

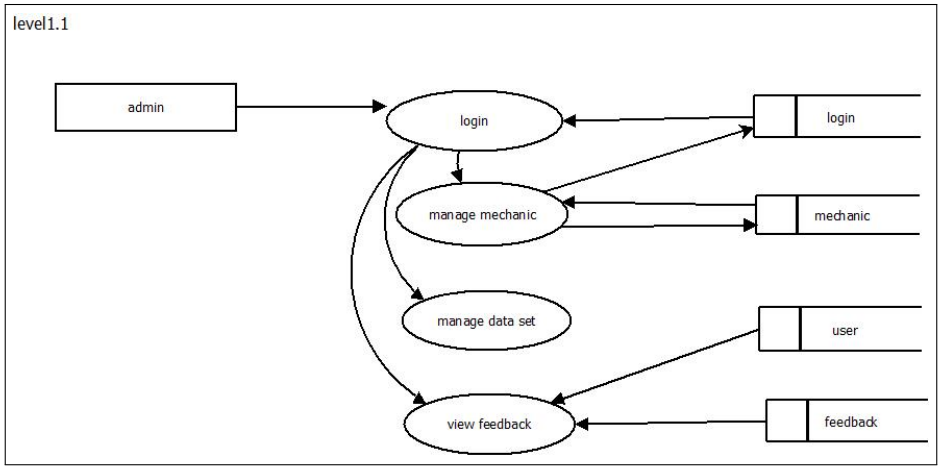


Figure 4.2: Level 1.1

4.2.3 LEVEL 1.2(USER)

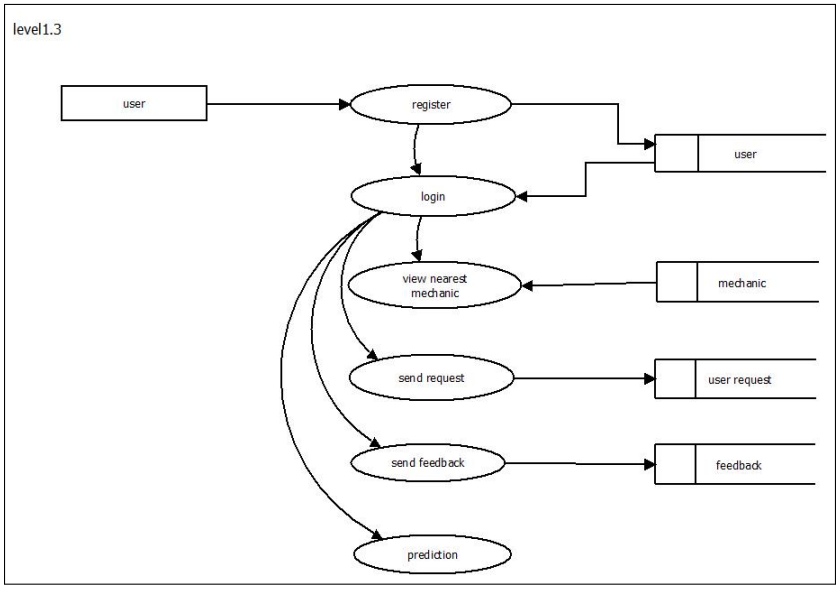


Figure 4.3: Level 1.2

4.2.4 LEVEL 1.3(MECHANIC)

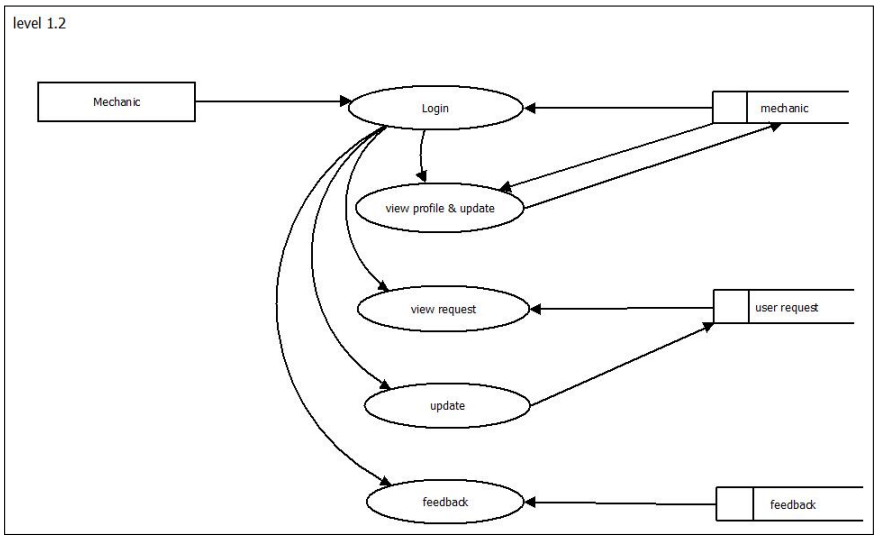


Figure 4.4: Level 1.3

4.2.5 LEVEL 1.4(FUEL)

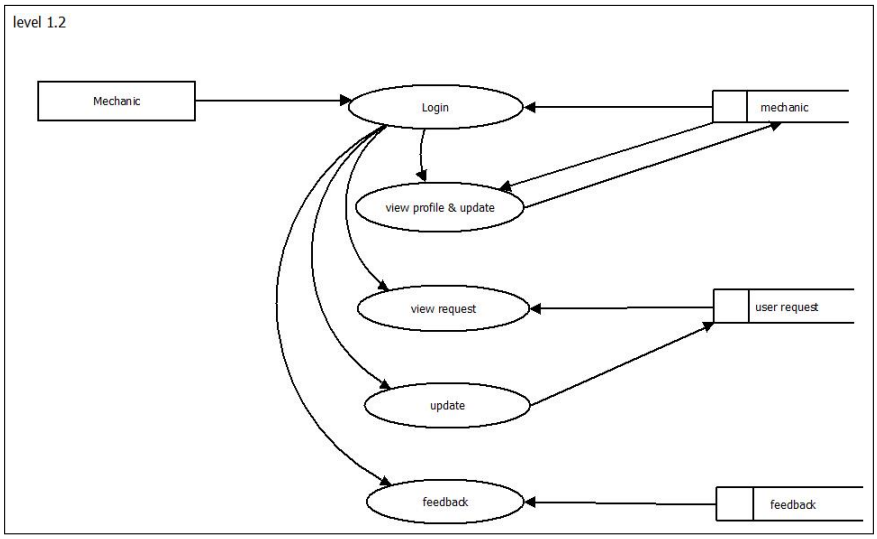


Figure 4.5: Level 1.4

4.2.6 LEVEL 1.4(FUEL)

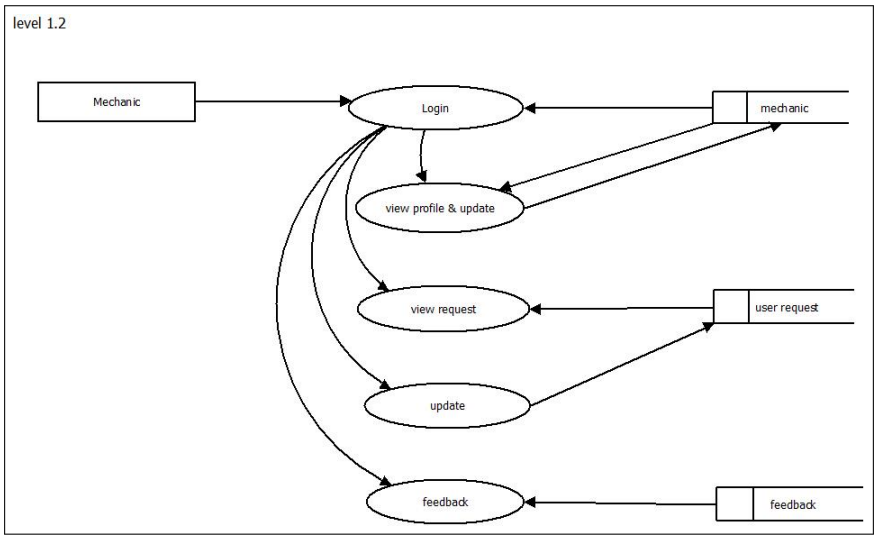


Figure 4.6: Level 1.4

4.3 DATA DATABASE DESIGN

4.3.1 Login

1 Messages | 2 Table Data | 3 Info

lid	user_name	password	type
1	admin	admin123	admin
2	raheem	123	user
3	RAHEEMUDHEENmec	123	mechanic
5	anas	123	user
6	siyasmec	123	mechanic
7	sanu	123	user
10	indian	123	fuelpump
12	fuel1	123	fuelpump
13	hp	123	fuelpump
14	hp	123	fuelpump
20	user2	123	user
21	user1	123	user

Database: vehicle_app Table: login

Figure 4.7: login

4.3.2 Location

1 Messages | 2 Table Data | 3 Info

loc_id	lid	latitude	longitude
1	2	11.376029	76.0208763
2	3	11.16763204	76.12503577
3	4	50.36769773	80.02498409
4	5	11.376029	76.0208763
5	6	11.3676953	76.02499177
6	7	11.36769343	76.02499223
7	10	11.36768431	76.02500566
8	8	11.376029	76.0208763
9	11	12.36768693	78.02500513
10	20	11.376029	76.0208763
11	21	11.2580535	75.7834772
12	22	11.2557588	75.7860203

Database: vehicle_app Table: location

Figure 4.8: location

4.3.3 Mechanic request

1 Messages | 2 Table Data | 3 Info

r_id	name	place	post	pin	phone	email	username	password
106	jithin	payyoli	payyoli	673529	9745706209	jithin123@gmail.com	jithin	123
107	ashif	thikkodi	thikkodi	673521	9745706207	ashifop@gmail.com	ashif	123
108	karthik	westhill	westhill	673528	9745706206	karthik@gmail.com	karthik	123
(Auto)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)

Figure 4.9: Mechanic request

4.3.4 mechanic

1 Messages | 2 Table Data | 3 Info

m_id	l_id	name	place	post	pin	phone	email
1	3	RAHEEMUDHEENmec	koodaranjiii	thiruvambady	673603	8113044223	raheemudheenmal182@gmail.com
3	6	siyasmec	tdy	tdy	765456	98776543321	hahaja@gmaol
4	27	ramesh	kadav	omassery	87654	9876543210	rajesg@gamail.com
5	28	amal	kkakyam	kakkayam	687656	9876567899	amal@
6	29	gautham	payyoli	payyoli	765443	8113044223	hshja@@
7	31	mecl	hahha	kkdjdj	786541	6113044223	tatagb@
9	39	farhan	pambizhanjapara	thiruvambady	673603	9874563210	raheemudheenmal182@gmail.com
10	40	abdhu	koduvally	kuduvally	765456	9876554321	kdy@
(Auto)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)

Figure 4.10: mechanic

4.3.5 Work request

1 Messages | 2 Table Data | 3 Info

r_id	ml_id	ul_id	work	date	status
1	3	2	my pologt i	2023-05-06	pending
2	3	2	access down	2023-05-06	pending
3	4	5	maruthi went	2023-05-06	accepted
4	6	7	completely	2023-05-06	pending
5	3	20	come here	2023-05-07	accepted
6	6	20	come pls	2023-05-07	pending
7	3	20	come here p	2023-05-07	accepted
8	3	21	1234555	2023-05-07	accepted
9	3	22	Engine soun	2023-05-07	pending
10	28	21	sghgbnunkok	2023-05-08	pending
11	3	21	vbij	2023-05-09	pending
12	3	2	vgugguug6g	2023-05-09	accepted

Database: vehicle_app Table: u_request

Figure 4.11: Work request

4.3.6 Fuel Pump request

1 Messages | 2 Table Data | 3 Info

fr_id	name	phone	email	place	username	password
5	hpthikkodi	9745706201	hpthikkodi@gmail.com	thikkodi	hpthikkodi	123
6	bharathpayyoli	9876543291	bharathpayyoli@gmail.com	payyoli	bharathpayyoli	123
7	hpwesthill	9845342609	hpwesthill@gmail.com	westhill	hpwesthill	123
(Auto)	*(NULL)*	*(NULL)*	*(NULL)*	*(NULL)*	*(NULL)*	*(NULL)*

Figure 4.12: Fuel Pump request

4.3.7 Fuel Pump

1 Messages | 2 Table Data | 3 Info

f_id	l_id	name	place	phone	email
2	10	indianoil	Thiruvambady	7890654432	abc@gmail
4	12	fuell	abc	8768906541	hqhq@gmail
5	13	hpmukkam	mukkam	9876543211	haa@gmail.c
8	33	nayarakkd	9999	788	9898989899
9	37	abcfuel	omassery	987456321	abcfuel@gam
10	41	nayara	kattangal	9898989898	nayara@gmai
11	42	fuell	omassery	8978675645	hah@hhw
(Auto)	*(NULL)*	*(NULL)*	*(NULL)*	*(NULL)*	*(NULL)*

Figure 4.13: Fuel Pump

4.3.8 Fuel Delivery request

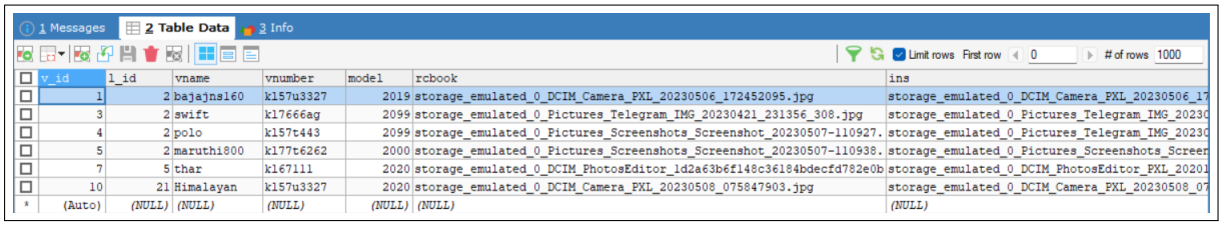
1 Messages | 2 Table Data | 3 Info

fdelv_id	f_id	l_id	fuel_type	quantity	date	status
1	11	20	diesel	11	2023-05-07	pending
2	11	21	diesel	11	2023-05-07	pending
3	10	21	petrol	11	2023-05-07	pending
4	11	21	petrol	11	2023-05-07	pending
5	10	21	diesel	11	2023-05-09	pending
6	10	21	petrol	11	2023-05-09	pending
7	10	21	petrol	11	2023-05-10	pending
8	8	21	petrol	11	2023-05-10	pending
9	8	21	diesel	11	2023-05-10	pending
10	13	21	diesel	11	2023-05-10	pending
11	13	21	petrol	5	2023-05-10	pending
12	8	2	diesel	4	2023-05-10	pending

Database: vehicle_app Table: fueldelivaryreq

Figure 4.14: Fuel Delivery request

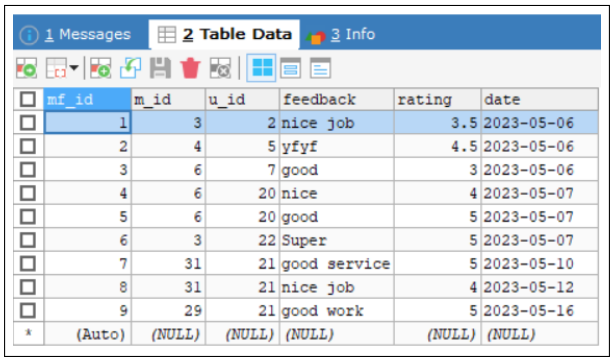
4.3.9 Vehicles



v_id	l_id	vname	vnumber	model	rcbook	ins
1	2	bajajns160	kl57u3327	2019	storage_emulated_0_DCIM_Camera_PXL_20230506_172452095.jpg	storage_emulated_0_DCIM_Camera_PXL_20230506_17
3	2	swift	kl7666ag	2099	storage_emulated_0_Pictures_Telegram_IMG_20230421_231356_308.jpg	storage_emulated_0_Pictures_Telegram_IMG_2023
4	2	polo	kl57t443	2099	storage_emulated_0_Pictures_Screenshots_Screenshot_20230507-110927.	storage_emulated_0_Pictures_Telegram_IMG_2023
5	2	maruthi800	kl77t6262	2000	storage_emulated_0_Pictures_Screenshots_Screenshot_20230507-110938.	storage_emulated_0_Pictures_Screenshots_Screen
7	5	thar	kl67111	2020	storage_emulated_0_DCIM_PhotosEditor_1d2a63b6f148c36184bdecfd782e0b	storage_emulated_0_DCIM_PhotosEditor_PXL_2020
10	21	Himalayan	kl57u3327	2020	storage_emulated_0_DCIM_Camera_PXL_20230508_075847903.jpg	storage_emulated_0_DCIM_Camera_PXL_20230508_07
(Auto)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)

Figure 4.15: Vehicles

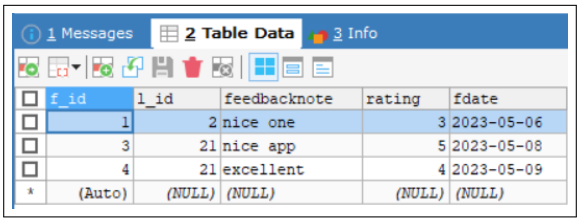
4.3.10 Mechanic Feed back



mf_id	m_id	u_id	feedback	rating	date
1	3	2	nice job	3.5	2023-05-06
2	4	5	yfyf	4.5	2023-05-06
3	6	7	good	3	2023-05-06
4	6	20	nice	4	2023-05-07
5	6	20	good	5	2023-05-07
6	3	22	Super	5	2023-05-07
7	31	21	good service	5	2023-05-10
8	31	21	nice job	4	2023-05-12
9	29	21	good work	5	2023-05-16
(Auto)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)

Figure 4.16: Mechanic Feed back

4.3.11 App Feed back



f_id	l_id	feedbacknote	rating	fdate
1	2	nice one	3	2023-05-06
3	21	nice app	5	2023-05-08
4	21	excellent	4	2023-05-09
(Auto)	(NULL)	(NULL)	(NULL)	(NULL)

Figure 4.17: App Feed back

4.4 FORM DESIGN

4.4.1 Web & Android UI

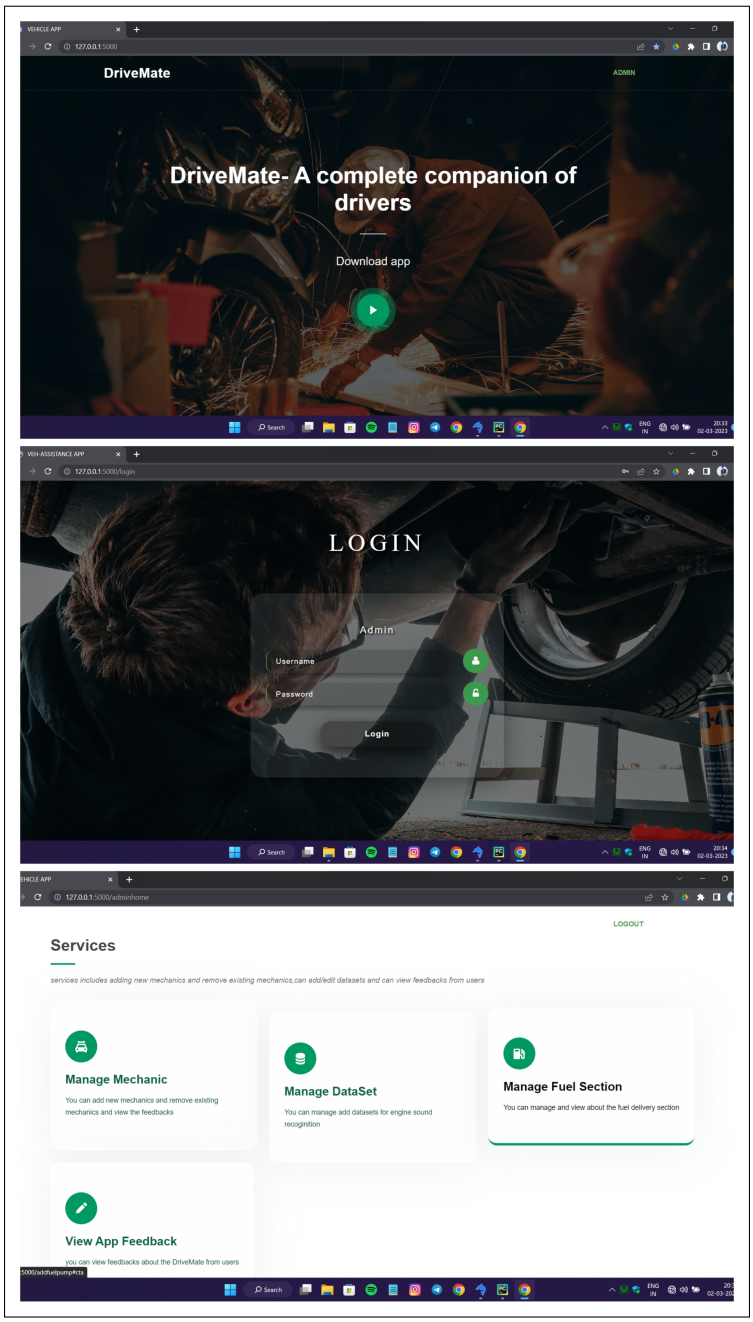


Figure 4.18: WEB interfaces1

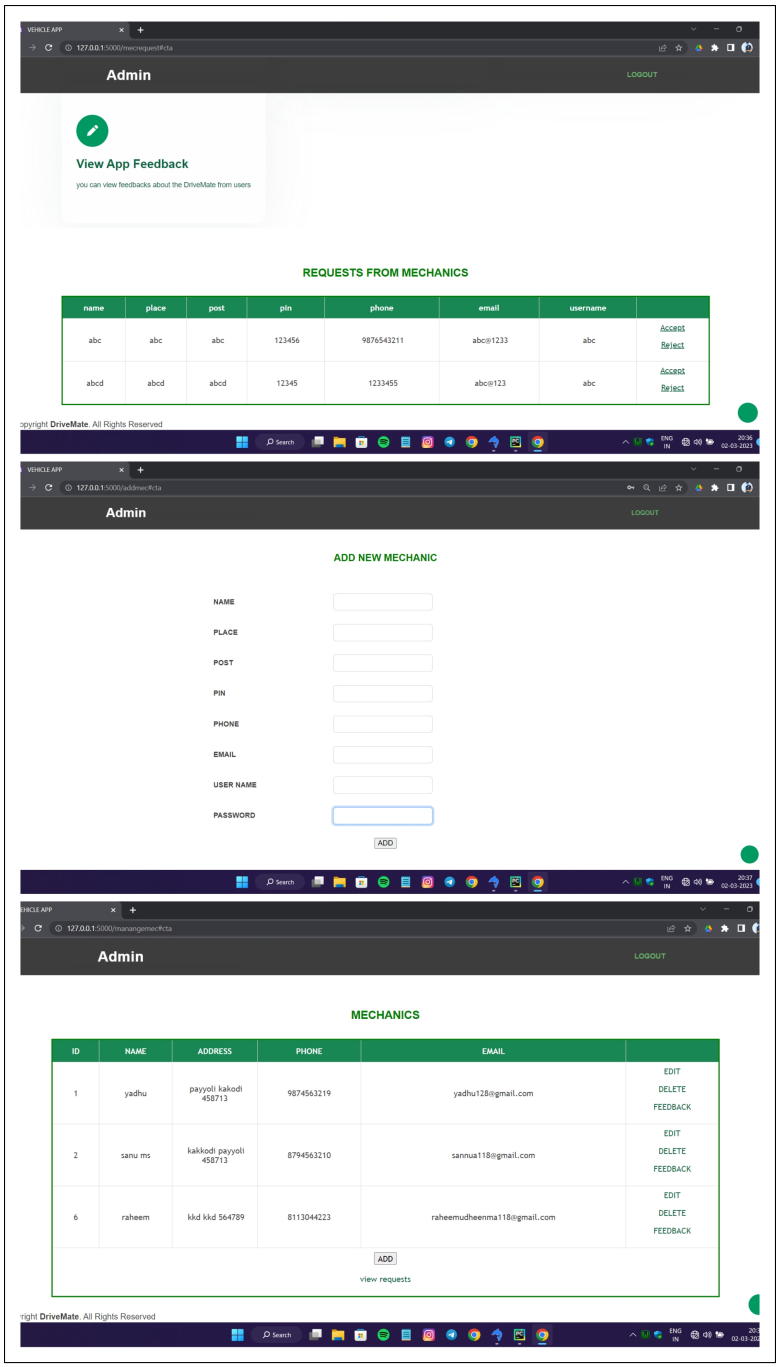


Figure 4.19: WEB interfaces2

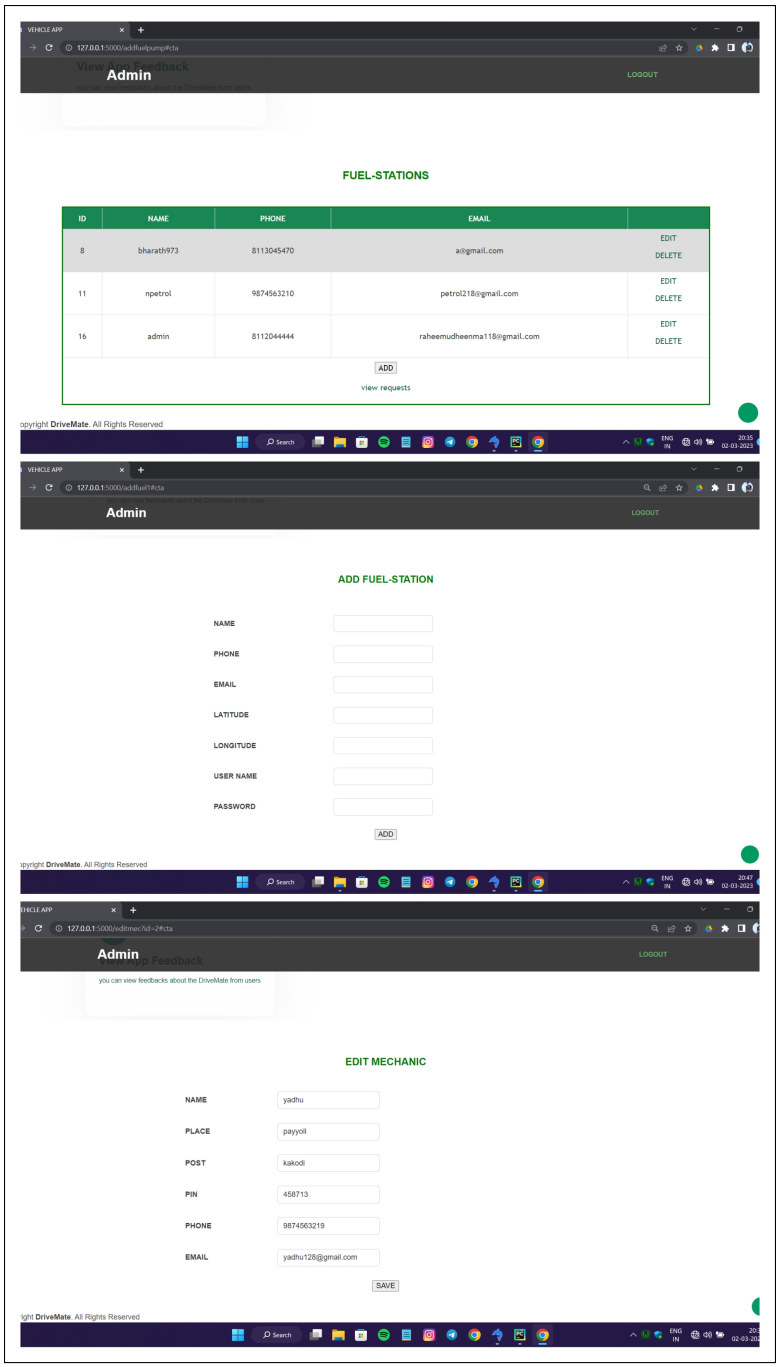


Figure 4.20: WEB interfaces3

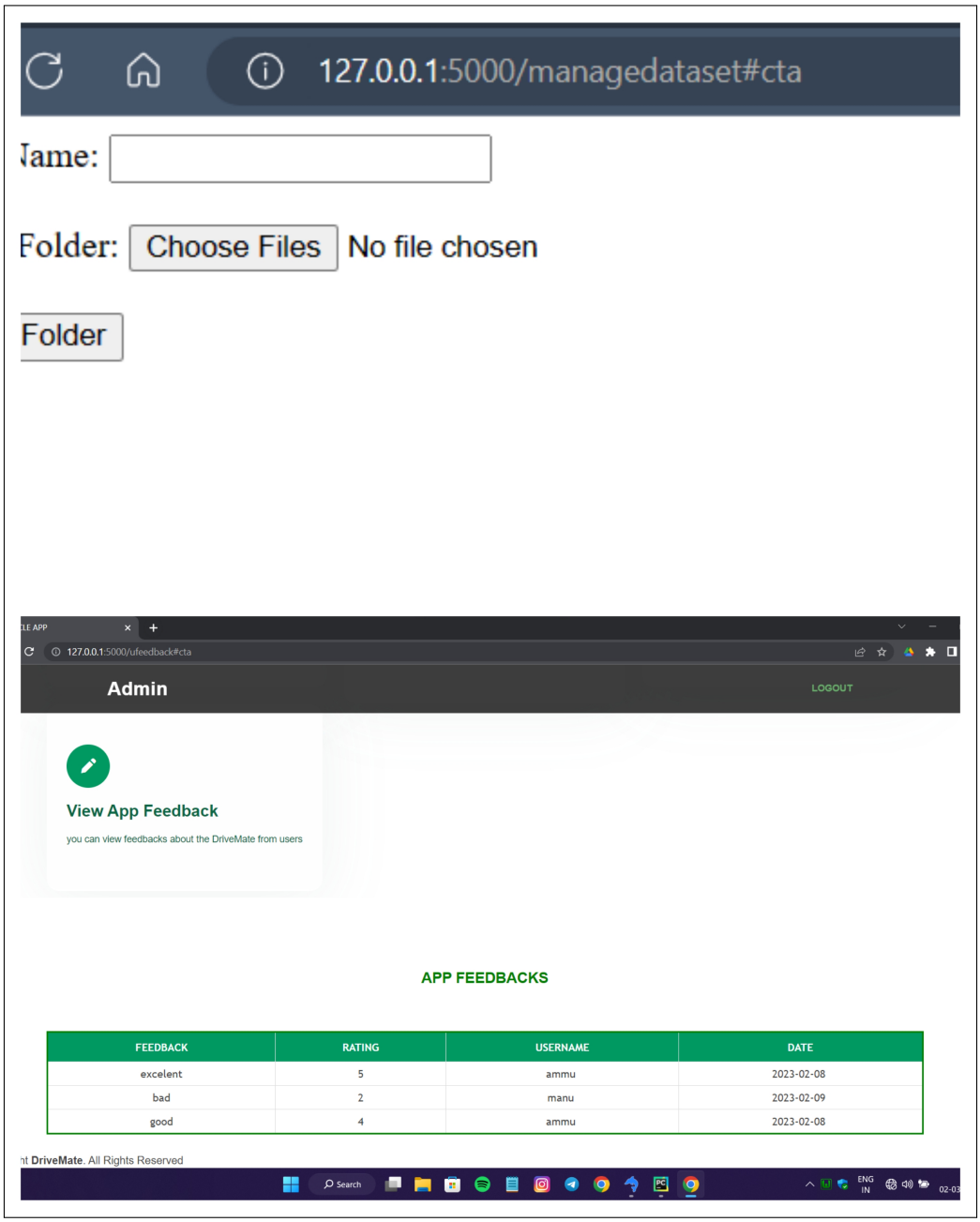


Figure 4.21: WEB interfaces4

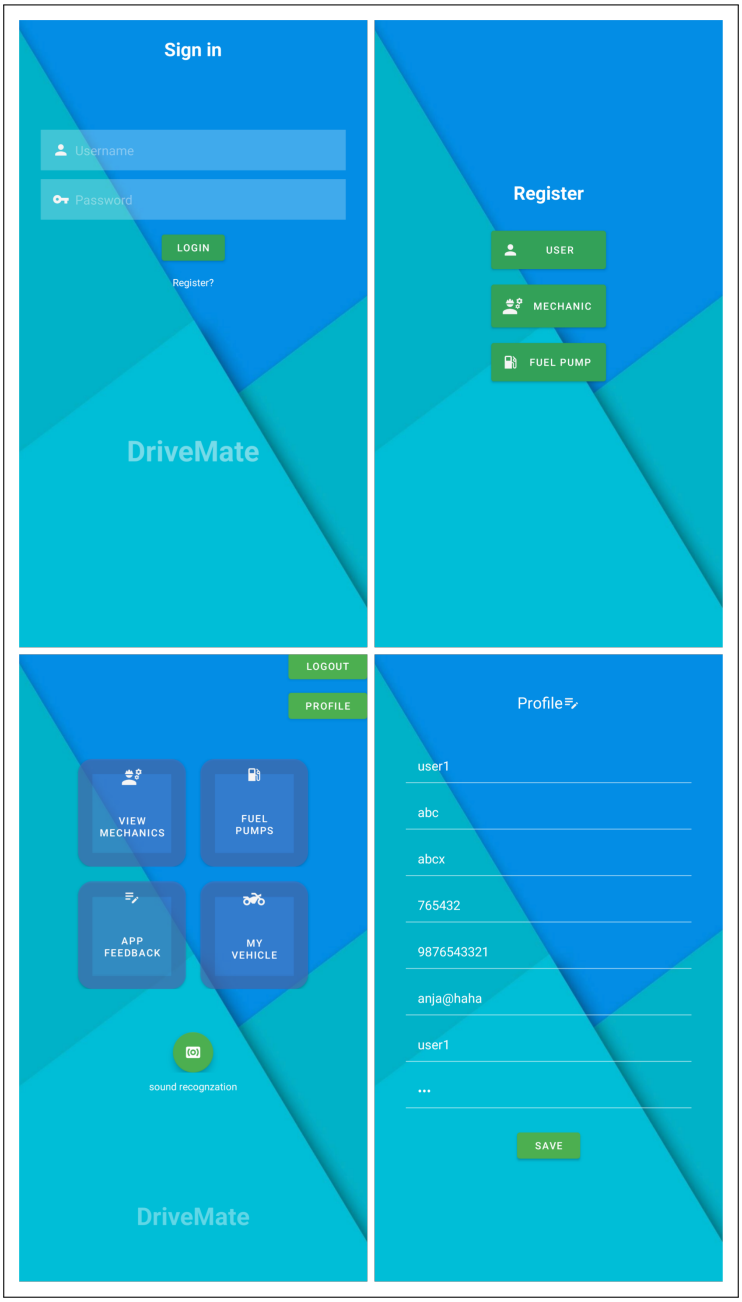


Figure 4.22: Android interfaces 1

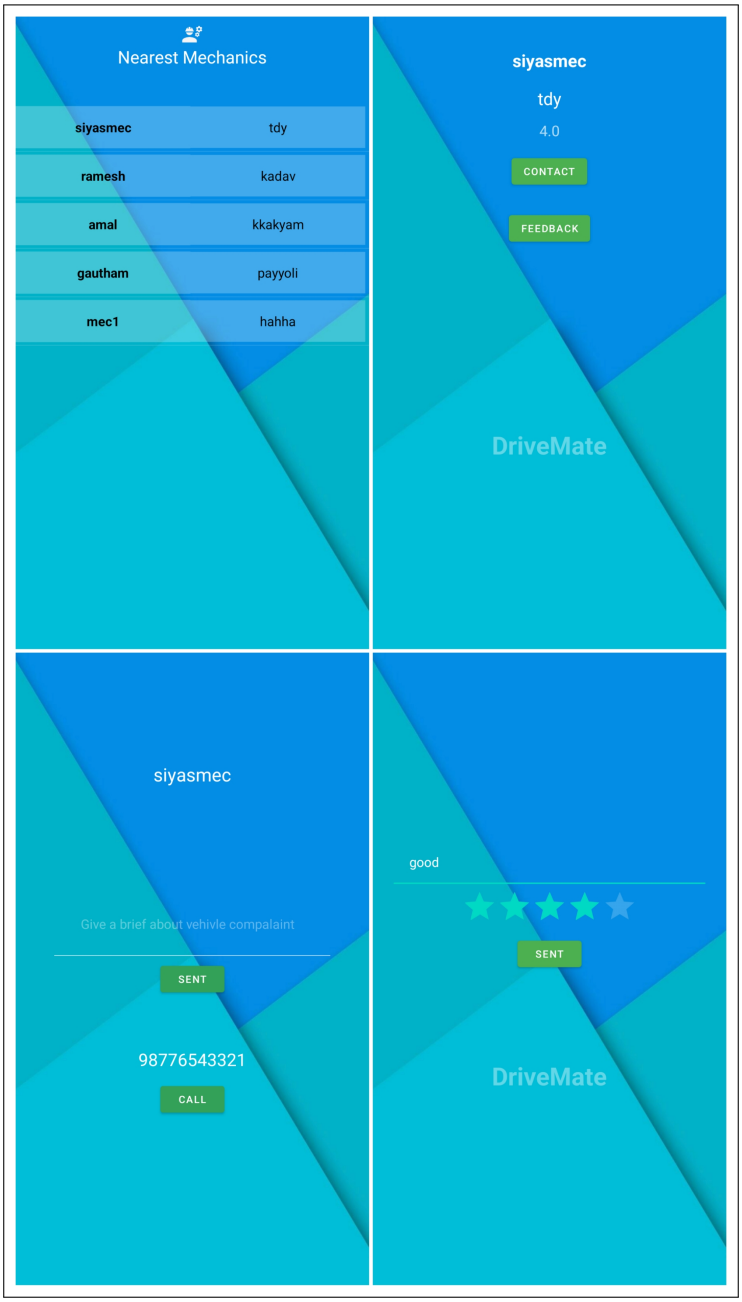


Figure 4.23: Android interfaces 2

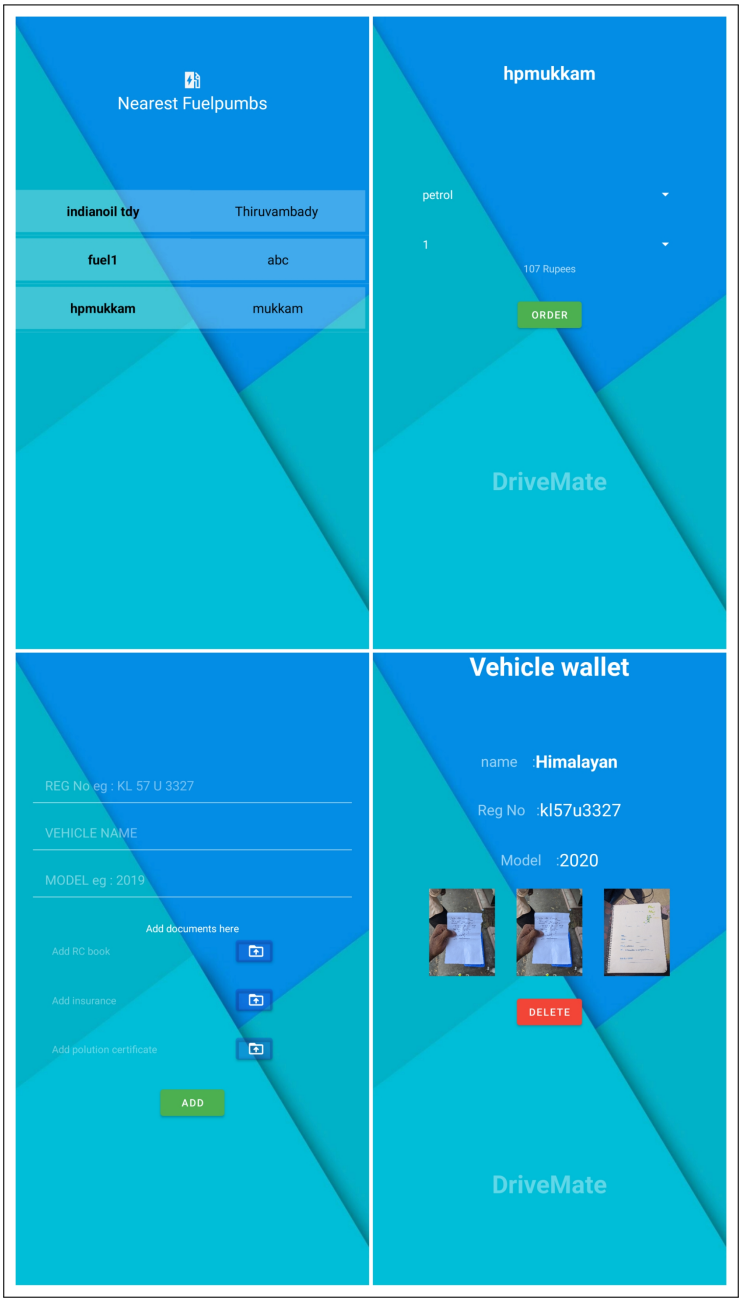


Figure 4.24: Android interfaces 3

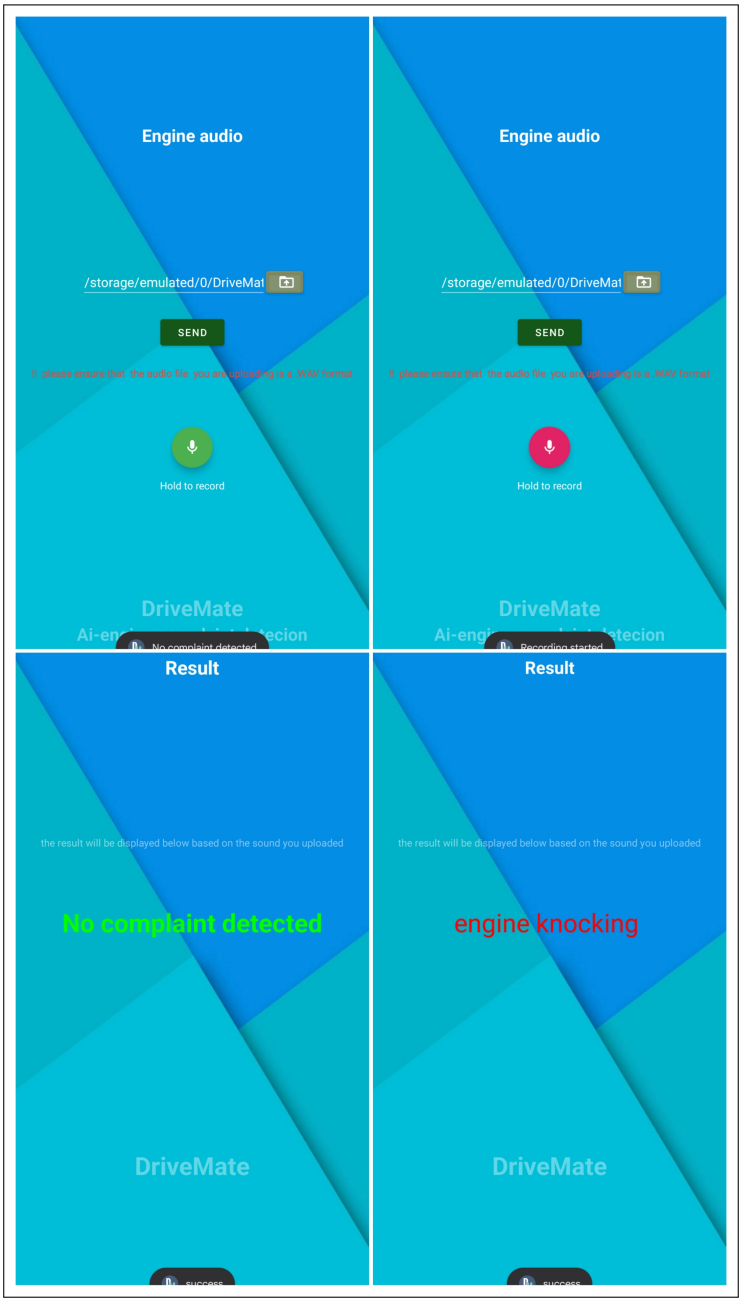


Figure 4.25: Android interfaces 4

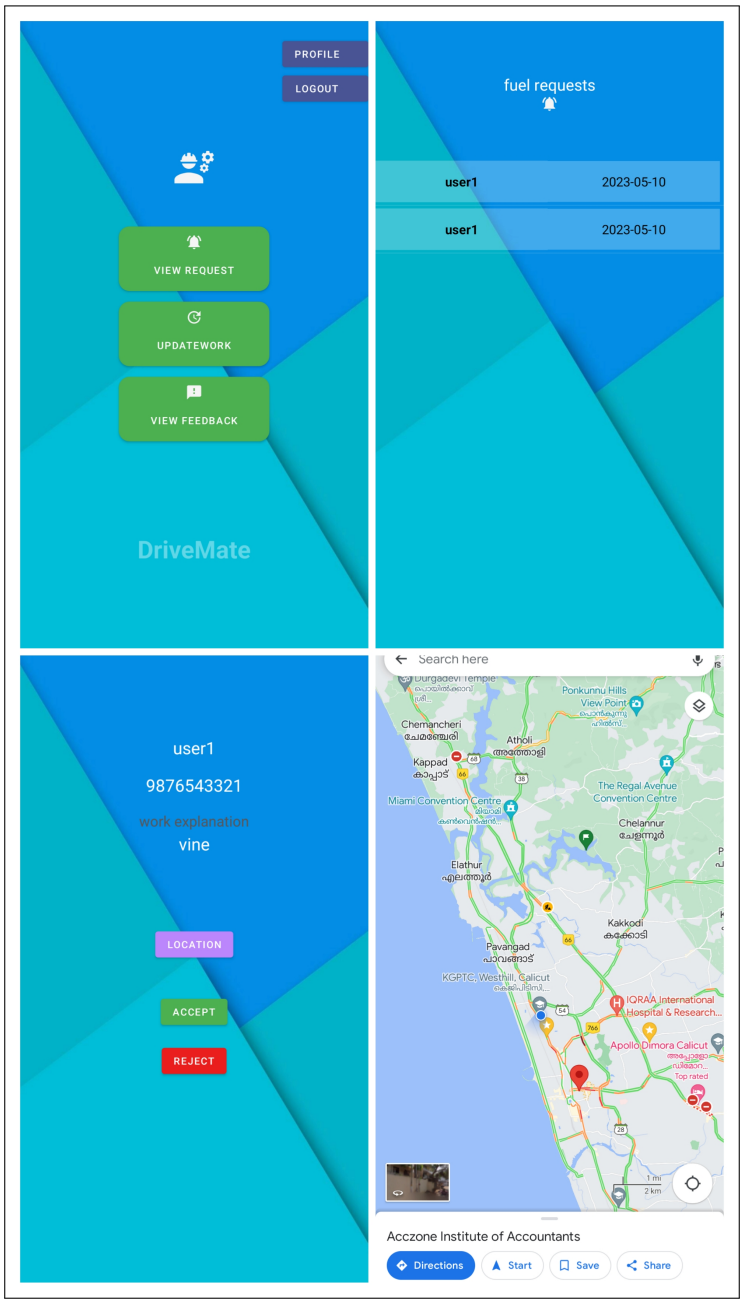


Figure 4.26: Android interfaces 5



Figure 4.27: Android interfaces 6

CHAPTER 5

SYSTEM IMPLEMENTATION

5.1 Methodology

In this section, we describe the methodology employed in the development of the Drive Mate project. It encompasses the utilization of the K-Nearest Neighbors (KNN) algorithm and the Mel-Frequency Cepstral Coefficients (MFCC) technique.

5.1.1 Mel-Frequency Cepstral Coefficients (MFCC) Technique

The Mel-Frequency Cepstral Coefficients (MFCC) technique is commonly employed in speech and audio processing to extract relevant features that capture the spectral characteristics of the signal. It involves a series of steps to transform the audio data into a compact representation suitable for analysis.

Application in Drive Mate

In the Drive Mate project, an Android application is developed to analyze vehicle engine sounds and predict engine complaints using the MFCC (Mel-Frequency Cepstral Coefficients) feature extraction technique. The application takes input in the form of vehicle engine sounds recorded in .wav format.The MFCC technique is applied to extract relevant features from the engine sounds. Firstly, the audio signals are pre-emphasized to enhance the high-frequency content. The pre-emphasized signals are then divided into short frames to capture short-term variations. The power spectrum of each frame is calculated using the Fast Fourier Transform (FFT), providing information about the energy distribution across different frequencies.To model the

non-linear perception of human hearing, a Mel filterbank is applied to the power spectrum. This filterbank consists of triangular filters spaced on the Mel scale, which approximates the human ear's frequency response. By multiplying the filterbank with the power spectrum, the technique derives Mel-scale filterbank energies that summarize the spectral content of the audio signals.To reduce dimensionality and emphasize important information, the logarithm of the filterbank energies is computed, compressing the dynamic range. Subsequently, the Discrete Cosine Transform (DCT) is applied to decorrelate the coefficients, resulting in a compact set of MFCC coefficients. These MFCC coefficients capture essential spectral features of the engine sounds, including characteristics related to engine complaints. By utilizing the KNN (K-Nearest Neighbors) algorithm, the application predicts engine complaints based on the calculated MFCC coefficients. The KNN algorithm considers the similarity between the test engine sound and the training data, selecting the K nearest neighbors and assigning the majority class label to the test sound.The integration of MFCC feature extraction into the Drive Mate application enables automated analysis and prediction of engine complaints based on the input engine sounds. This allows users to quickly assess potential issues with their vehicles, facilitating timely maintenance and repairs.

5.1.2 K-Nearest Neighbors (KNN) Algorithm

The K-Nearest Neighbors (KNN) algorithm is a non-parametric classification method widely used for pattern recognition and machine learning tasks. It operates on the principle of finding the K nearest data points in the feature space to a given input and assigning a label based on the majority vote of the neighbors.

Application in Drive Mate

In the Drive Mate application, the KNN (K-Nearest Neighbors) algorithm is applied for engine complaint prediction based on the extracted MFCC (Mel-Frequency Cepstral Coefficients) features from vehicle engine sounds. The KNN algorithm is well-suited for this task as it leverages the similarities between instances to make predictions. After extracting the MFCC coefficients from the input engine sound, the KNN algorithm is employed to find the K

nearest neighbors from the pre-labeled training dataset. These neighbors are determined based on the distances or similarities calculated using a distance metric such as Euclidean distance or cosine similarity. The KNN algorithm then applies a majority voting scheme to determine the predicted engine complaint for the input sound. Each of the K nearest neighbors contributes one vote, and the class label with the highest number of votes is assigned as the predicted complaint. This approach ensures that the prediction is based on the consensus of the nearest neighbors, taking into account the collective information from similar instances.By utilizing the KNN algorithm within the Drive Mate application, users can benefit from the engine complaint prediction functionality. This enables them to quickly identify potential issues with their vehicles based on the analysis of engine sounds, facilitating proactive maintenance and timely resolution of engine-related problems.

5.2 Important Codes

5.2.1 Web codes

```
\begin{lstlisting}

from flask import *

from src.db import *

app=Flask(__name__)
app.secret_key="1233"

@app.route('/',methods=['post','get'])
def home():
    return render_template("HOME.html")
@app.route('/login')
def login():
    return render_template("loginIndex.html")
@app.route('/adminhome',methods=['get','post'])
```

```
def adminhome():
        return render_template("adminindex.html")

@app.route('/manangemec',methods=['post','get'])
def managemec():
    qry=" select * from mechanic"
    res=selectall(qry)
    return render_template("MANGE MEC.html",val=res)
@app.route('/mecrequest',methods=['post','get'])
def mecrequest():
    qry="select * from mecrequest"
    res = selectall(qry)
    return render_template("mecrequest.html",val=res)
@app.route('/reject',methods=['post','get'])
def reject():
    id2=request.args.get('rid')
    qry="delete from mecrequest where r_id=%s"
    iud(qry, id2)
    return '''<script>alert("request from mechanic
    rejected successfully"); window.location
    '/mecrequest#cta'</script>'''
@app.route('/accept',methods=['post','get'])
def accept():
    id3 = request.args.get('rid')
    session['rid'] = id3
    qry = "select * from mecrequest where r_id=%s"
    res = selectone(qry, id3)
    name=res['name']
    place=res['place']
    post = res['post']
    pin = res['pin']
```

```
    phone = res['phone']
    email = res['email']
    username = res['username']
    password = res['password']
    qry = "insert into login values(null,%s
    ,%s,'mechanic')"
    val = (username, password)
    id3 = iud(qry, val)
    qry = "insert into mechanic values(null
    ,%s,%s,%s,%s,%s,%s,%s)"
    val = (id3, name, place, post, pin,
    phone, email)
    iud(qry, val)
    id2 = request.args.get('rid')
    qry = "delete from mecrequest where r_id=%s"
    iud(qry, id2)
    return '''<script>alert("succesfully
    added a mechanic"); window.location=
    '/mecrequest#cta'</script>''
@app.route('/addmec',methods=['post','get'])
def addmec():
    return render_template("ADD MEC.html")

@app.route('/reg',methods=['post','get'])
def reg():
    name=request.form['textfield']
    place= request.form['textfield2']
    post=request.form['textfield3']
    pin=request.form['textfield4']
    phone=request.form['textfield5']
    email=request.form['textfield6']
```

```
    username=request.form['textfield7']
    password=request.form['textfield8']
    qry="insert into login values(null
    ,%s,%s,'mechanic')"
    val=(username,password)
    id=iud(qry,val)
    qry="insert into mechanic values(null
    ,%s,%s,%s,%s,%s,%s,%s)"

@app.route('/delete',methods=['post','get'])
def delete():
    id=request.args.get('id')
    qry="delete from login where lid=%s"
    iud(qry,id)
    qry="delete from mechanic  where l_id=%s"
    iud(qry,id)
    return '''<script> window.location=
    '/manangemec#cta'</script>'''
@app.route('/managedataset',methods=['post','get'])
def managedataset():
   return render_template("managedataset.html")
```

5.2.2 complaint prediction-KNN algorithm (Prediction.py)

```
# import tflearn
import numpy as np
# from keras.callbacks import EarlyStopping, ModelCheckpoint

# import speechData
# import tensorflow as tf
# from sklearn.model_selection import train_test_split
```

```
# from sklearn.metrics import confusion_matrix
# from keras.utils.vis_utils import plot_model
import librosa
import os
res=os.listdir("static/dataset")
print(len(res))

import csv
mydict = []
# import matplotlib.pyplot as plt
# from matplotlib import pyplot

learning_rate = 0.00001
training_iters =80  # steps
#
width = 20  # mfcc features
height = 80  # (max) length of utterance
classes = 12  # digits



from collections import Counter


class CKNN:

    def __init__(self):
        self.accurate_predictions = 0
        self.total_predictions = 0
        self.accuracy = 0.0
        ##########
```

```
        with open('labels1.data') as f:
            lines = f.readlines()
        lines=[s.strip('\n') for s in lines]
        print(lines)
        training_data=np.loadtxt
        ("sample1.data",dtype=float,delimiter=" ")

        self.training_set= {'0':[],'1':[],'2':[],
        '3':[],'4':[],'5':[],'6':[],'7':[],'8':[],
        '9':[],'10':[],'11':[],'12':[],}
        cnt=0

        for record in training_data:
            st=lines[cnt][0]
            cnt+=1



            self.training_set[st[-1]].append( record[:])

    def predict(self,  to_predict, k = 1):



        distributions = []
        for group in self.training_set:
            i=0
            # print(group,'group')
            for features in self.training_set[group]:

                euclidean_distance = np.linalg.norm
                (np.array(features)- np.array(to_predict))
```

```
                distributions.append([euclidean_distance, group])

        # print(distributions)
        results = [i[1] for i in sorted(distributions)[:k]]
        result = Counter(results).most_common(1)[0][0]
        # print("rs",results,self.training_set.keys())
        confidence = Counter(results).most_common(1)[0][1]/k

        return result, confidence


def prep1(filename):

    knn = CKNN()
    #print(filename)
    feat=[filename]
    res=knn.predict(feat)
    print(res[0])
    print(res[1],"111============
    ==============================")
    return res[0]
def mfcc_target1(predict):
    batch_features = []
    files = [predict]
    #load audio file
    for wav in files:
        if not wav.endswith(".wav"): continue
        wave, sr = librosa.load(predict, mono=True)
        #feature extraction
        mfcc = librosa.feature.mfcc(wave, sr)
        print("liborsa Mfcc = ",np.array(mfcc).shape)
```

```
        print("features are :",mfcc)
        print("==================++++++++++
        ++++++++++++++++++++++++++=")
        lenn = 0
        if len(mfcc[0]) < 500:

            lenn = len(mfcc[0])

        mfcc = np.pad(mfcc, ((0, 0),
        (0, 500 - lenn)), mode='constant',
        constant_values=0)
        # samples = []
        # for i in range(0, 20):
        #     row = []
        #     for j in range(80):
        #         row.append(mfcc[i][j])
        #
        #     samples.append(row)
        batch_features.append(np.array(mfcc))

    print(np.array(batch_features).shape)
    return batch_features


def userpredict(wav):
    print(wav)
    row = []
    feature = mfcc_target1(wav)
    ---------------------------------
    ------------',len(feature))
    for ii in feature[0]:
```

```
        # print(len(ii), ii, "==========
        ========")
        avg = sum(ii) / len(ii)
        row.append(avg)
    print(len(row), "==========
    =========")
    return prep1(row)
```

5.2.3 web database connection (db.py)

```
import pymysql
def iud(qry,val):
con=pymysql.connect(host='localhost'
,port=3306,user='root',password='119881141',db='vehicle_app')
    cmd=con.cursor()
    cmd.execute(qry,val)
    id=cmd.lastrowid
    con.commit()
    con.close()

    return id

def selectone(qry,val):
    con=pymysql.connect(host='localhost',port=3306
    ,user='root',password='119881141',db='vehicle_app'
    ,cursorclass=pymysql.cursors.DictCursor)
    cmd=con.cursor()
    cmd.execute(qry,val)
    res=cmd.fetchone()

    return res
```

```
def selectall(qry):
    con=pymysql.connect(host='localhost',port=3306,
    user='root',password='119881141',db='vehicle_app',
    cursorclass=pymysql.cursors.DictCursor)
    cmd=con.cursor()
    cmd.execute(qry)
    res=cmd.fetchall()
    return res
def selectall2(qry,val):
    con=pymysql.connect(host='localhost',port=3306,
    user='root',password='119881141',db='vehicle_app',
    cursorclass=pymysql.cursors.DictCursor)
    cmd=con.cursor()
    cmd.execute(qry,val)
    res=cmd.fetchall()
    return res
```

5.2.4 Training Data (speech.py)

```
import os
import numpy as np
import librosa
from random import shuffle




# // saving/loading data set
def loadDataSet():
    samples = []
    names = []

    X, Y = mfcc_generator()
    print(X[0])
```

```
    print(Y[0])
    for i in range(0,len(Y)):
        row=[]
        for ii in X[i]:
            print(len(ii),ii,"=========
            ==============")
            avg=sum(ii)/len(ii)
            row.append(avg)
        samples.append(row)
        names.append(Y[i])
    np.savetxt("sample1.data",samples)
    np.savetxt("labels1.data",names)

def mfcc_generator():

    batch_features = []
    labels = []

    filess=["highidlerpm-activa",
    "highidlerpm-activa125","highidlerpm-ns160",
    "invalid","knoking-activa", "knoking-activa125"
    ,"loweridlerpm-activa","loweridlerpm-activa125",
    "loweridlerpm-ns160",  "normal-activa",
    "normal-activa125","normal-ns160","tappetissue-ns160"]

    for i in range(0,len(filess)):
        files = os.listdir(path+filess[i])
        shuffle(files)

        for wav in files:
            if not wav.endswith(".wav"): continue
```

```
            try:
                print("========================")
                wave, sr = librosa.load(path
                +filess[i]+"/"+ wav, mono=True)
                mfcc = librosa.feature.mfcc(wave, sr)
                lenn=0
                if len(mfcc[0])<500:
                    lenn=len(mfcc[0])



            except Exception as e:
                print(e,"============================"
                ,path +filess[i]+"/"+ wav)
                print(path +filess[i]+"/"+ wav)

    # print(np.array(batch_features).shape)
    return batch_features, labels

loadDataSet()
```

5.2.5 Android-data base connection (DataBasehelper.java)

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DatabaseHelper {

    private static Connection connection = null;
    private static final String url =

    public static Connection getConnection() {
        try {
```

```
            Class.forName("com.mysql.jdbc.Driver");
            connection = DriverManager.getConnecti
            on(url, user, password);
        } catch (ClassNotFoundException e) {
            e.printStackTrace();
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return connection;
    }
}
```

5.2.6 MAinActivity(MAinActivity.java)

```
import androidx.activity.result.contract.ActivityResultContracts;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.NotificationCompat;
import androidx.core.app.NotificationManagerCompat;
import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.app.PendingIntent;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;
import android.content.Context;
import android.content.Intent;
import android.location.LocationManager;
import android.provider.Settings;
import androidx.core.app.NotificationCompat;
import androidx.core.app.NotificationManagerCompat;
import android.app.PendingIntent;
```

```
public class MainActivity extends AppCompatActivity {

    @Override
    protected void onResume() {

        super.onResume();

    // Inside onResume() method of your main activity
    @Override
    public void onBackPressed() {
        finishAffinity();
    }
    Button b1;
    SharedPreferences sh;
    EditText e1;
    String ip;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
  NotificationChannel channel = new NotificationChannel
  ("default", "Default channel", NotificationManager
  .IMPORTANCE_HIGH);
NotificationManager notificationManager
        }

b1=findViewById(R.id.sentbtn);
    shPreferenceManager.getDefaultSharedPreferences
    (getApplicationContext());
        e1=findViewById(R.id.ip);
```

```
        e1.setText(sh.getString("ip",""));
        b1.setOnClickListener(new View.OnClickListener()
        {
            @Override
            public void onClick(View v) {

                (getApplicationContext(), "default")
 .setSmallIcon(R.drawable.notifydm)
 .setContentTitle("DriveMate")
 .setContentText("Turn on the
  location for the proper working of DriveMate")
 .setPriority(NotificationCompat.PRIORITY_HIGH)
.setCategory(NotificationCompat.CATEGORY_CALL)
.setCategory(NotificationCompat.CATEGORY_MESSAGE);

 // Show the notification

  notificationManager.notify(1, builder.build());
p=e1.getText().toString();
 SharedPreferences.Editor edp = sh.edit();
edp.putString("ip",ip);
 edp.commit();

Intent intent=new Intent(getApplicationContext(),login.class);
startActivity(intent);

            }
        });
    }
}
```

CHAPTER 6

SYSTEM TESTING

6.1 UNIT TESTING

SI NO	TEST CASE	INPUT	EXPECTED OUTPUT	ACTUAL OUTPUT	REMARKS
1	Test case for adding valid login credentials	Valid user name& password	Redirect to home page	Redirect to home page	PASS
2	Test Case for adding invalid login credentials	invalid user name& password	Do not Redirect to home page	Do not Redirect to home page	PASS
3	Test case for User Registration	Valid name, place, phone, pin, Email, username, password,	Redirect to login page	Redirect to login page	PASS
4	Test case for Mechanic Registration	Valid mechanic name, place, phone, pin, Email, username, password,	Redirect to login page	Redirect to login page	PASS
5	Test case for Fuel pump Registration	Valid Fuel pump name, place, phone, Email, username, password,	Redirect to login page	Redirect to login page	PASS
6	Test case for editing profile	Valid mechanic name, place, phone, pin, Email, username, password,	Pop up-" profile edited successfully" & Redirect to home page	Pop up-" profile edited successfully" & Redirect to home page	
7	Test case for User giving feed back	Rating star & valid comment	Pop up- "feed back send successfully"	Pop up- "feed back send successfully"	PASS

6.2 INTEGRATION TESTING

SI NO	TEST CASE	INPUT	EXPECTED OUTPUT	ACTUAL OUTPUT	REMARKS
1	Finding nearest Mechanic	User's GPS location	View List of nearest mechanic	View List of nearest mechanic	PASS
2	Finding nearest Fuel pump	User's GPS location	View List of nearest Fuel pump	View List of nearest Fuel pump	PASS
3	Adding user's vehicle details to document wallet	Vehicle registration number, model year , vehicle name, images of vehicle documents	Pop up saying "vehicle added successfully"	Pop up saying "vehicle added successfully"	PASS
4	Engine complaint prediction using upload file option	.wav audio file	Showing result of prediction	Showing result of prediction	PASS
5	Engine complaint prediction using live recording button	Smartphone microphone input	Showing result of prediction	Showing result of prediction	PASS

6.3 SYSTEM TESTING

SI	TEST CASE	INPUT	EXPECTED OUTPUT	ACTUAL OUTPUT	REMARKS
1	Load app with internet connection	None	App load perfectly	App load perfectly	PASS
2	Load app without internet connection	None	App didn't load	App didn't load	PASS
3	Load website internet connection	None	website load perfectly	website load perfectly	PASS
4	Load website without internet connection	None	website didn't load	website didn't load	PASS
5	Insert Data set	Folder with valid data set	Dialogue box saying" data set uploaded"	Dialogue box saying" data set uploaded"	PASS

.

CHAPTER 7

CONCLUSION

In conclusion, the proposed application offers a valuable solution to address the engine complaints commonly experienced in petrol and diesel engine vehicles, which are essential for daily transportation in our society. By utilizing an unsupervised deep learning algorithm (KNN) to detect abnormal sounds from IC engines, the system can predict engine failures at an early stage, enabling timely maintenance and repairs. Moreover, the application serves as an online mechanic locator, making it easier and quicker for users to find mechanics in different locations. This not only reduces the time and cost associated with searching for mechanics but also streamlines the process for both users and mechanics. Overall, the proposed system provides a practical and efficient approach to improving engine maintenance and enhancing the convenience of finding reliable mechanics, ultimately benefiting the majority of the population dependent on internal combustion engine vehicles in their daily lives.

CHAPTER 8

FUTURE SCOPES

The future scopes of the DriveMate project are extensive and hold great potential for growth and advancement. DriveMate can expand its reach to a global market by adapting the application to different languages, regulations, and vehicle types, allowing it to cater to a broader user base worldwide. Integration with connected car technology and partnerships with vehicle manufacturers can elevate the capabilities of DriveMate, leveraging advanced data analytics, telematics systems, and onboard sensors for more accurate diagnostics and predictive maintenance. Additionally, DriveMate can explore collaborations with smart city initiatives, ride-sharing platforms, and car-sharing services to enhance its functionalities and provide a comprehensive smart mobility experience. The adoption of blockchain technology can bring enhanced data security and transparency to DriveMate, ensuring trust and user control over diagnostic data. With these future scopes, DriveMate is poised to expand its offerings, strengthen its market presence, and continue delivering valuable vehicle assistance services to drivers around the world.

CHAPTER 9

REFERENCES

[1] https://stackoverflow.com/

[2] https://chat.openai.com/

[3] https://www.python.org/community/

[4] https://source.android.com/

[5] https://github.com/